

DATA STRUCTURES

LAB MANUAL

Subject Code: CS1802

Class: I YEAR SEM-II (CSE)

Prepared By

Mr. G. Jaya Krishna, Assistant Professor, CSE



Department of Computer Science & Engineering

International Institute of Information Technology - Srikakulam
RGUKT A.P-532 402

DATA STRUCTURES LAB SYLLABUS

OBJECTIVE:

The objective of this lab is to teach students various data structures and to explain them algorithms for performing various operations on these data structures. This lab complements the data structures course. Students will gain practical knowledge by writing and executing programs in C using various data structures such as arrays, linked lists, stacks, queues, trees, graphs, hash tables and search trees.

OUTCOMES: Upon the completion of Data Structures practical course, the student will be able to:

1. **Identity** the appropriate data structure for given problem.
2. **Understand** the applications of data structures.
3. **Choose** the appropriate data structure and algorithm design method for a specified application.
4. **Understand** which algorithm or data structure to use in different scenarios.
5. **Understand** and apply fundamental algorithmic problems including Tree traversals, Graph traversals.
6. **Compare** different implementations of data structures and to recognize the advantages and disadvantages of them.
7. **Design and analyze** the time and space efficiency of the data structure.
8. **Write** complex applications using structured programming methods.

Lab Credits:

I YEAR SEM-II B.Tech CSE	LAB	L	T	P	C
CODE:CS1802	DATA STRUCTURES	0	0	3	2

S.No.	List of Experiments	Page. No
1	Recursion	
	1.1 write a Program to find GCD for given 2 numbers	6
	1.2 Write a Program to multiply 2 numbers without using * operator	6
	1.3 Write a Program to divide 2 numbers without using / operator	7
	1.4 write a Program to find power of the numbers (x, y)	8
	1.5 write a Program to Check Whether a Number is Palindrome or Not	9
	1.6 write a program for Tower of Hanoi problem	10
2.	Linked List	
	2.1 Write a Program to create a Singe Linked List and perform all insertion and deletion cases.	11
	2.2 write a Program to create a Circular Linked List and perform all insertion and deletion cases.	16
	2.3 write a Program to create a Double Linked List and perform all insertion and deletion cases	21
	2.4 write a Program to create a SLL and reverse all the links.	27
	2.5 write a Program to store polynomial in a linked list and apply add and subtract operations on two polynomials	29
	2.6 write a Program to sort the numbers given in a LL	32
	2.7 write a Program to create LL which stores details of students and print the name of the students who got first class	34
3.	Stacks	
	3.1 Write a Program to perform push, pop and peek operations on a stack using arrays.	36
	3.2 Write a Program to implement stack using Linked List.	38
	3.3 Write a Program to reverse the contents of stack.	40
	3.4 Write a Program to check nesting of parenthesis.	42
	3.5 write a Program to convert infix to postfix and prefix.	45
	3.6 write a Program to evaluate a postfix and prefix expressions.	49
4.	Queues	
	4.1 write a Program to implement queue using arrays	52
	4.2 write a Program to implement queue using Linked List	54
	4.3 write a Program to implement circular queue	55
	4.4 write a Program to implement queue which permits insertion ³ 112	57

	and deletion at both the ends using Arrays	
	4.5 write a Program to implement queue which permits insertion and deletion at both the ends using Linked List	60
5.	Binary Search Tree	
	5.1 write a Program to create BST and perform insertion, deletion and 3 traversal operations	62
	5.2 write a Program to count number of nodes of a BST	66
	5.3 write a Program to find nth node in the InOrder traversal of a BST	69
	5.4 write a Program to find largest and smallest elements of a BST	70
	5.5 write a Program to find all the elements of Nth level	73
	5.6 write a Program to find nodes which are at max distance from the root in a BST	75
6.	Heaps	
	6.1 write a Program to create MAX heap	78
	6.2 write a program to create MIN Heap	81
7.	Hashing	
	7.1 write a Program to search for an element in a hash table	84
Content Beyond Syllabi		
8	Write a program to insert a node after the nth node from the right end.	88
9	Write logic to delete all duplicate elements from single linked list	91
10	Write a program to sort and reverse after nth position in single linked list	93
11	Write a program to check whether entered string is a subsequence string of the existing string.	96
12	Write a program to multiply two polynomials	98
13	Write a program to check whether the given or entered string is palindrome or not, implement using Queue.	101
14	There are n persons who are arranged in circular fashion order (CLL). Find the winner after deleting k-th element respectively.	102
15	Design a calculator application by using either PREFIX or POSTFIX algebraic expression as internal Data structure. Note: Take the entire input from the user at a time.	105
16	There are 10 songs namely a,b,c,d,e,f,g,h,i,j when the user enter	107

	'n' letter you have to display the next song. Take default song as middle song. If the user enter 'p' you have to display the previous song. When the user enter the 'e' you have to exit from the music player. When user is at 10th song he/she can directly move to the first node. when user enter the 's' letter you have to provide option to search a song, if song exist play the song otherwise display song not found. If user wants to add new songs to the music player he/she can add by pressing the word 'a', if you want to delete a song you have to press the letter 'd' and select which song you want to delete, if that song exist delete it, if not exist display not exist. If user enter 'rs' letter (re-shuffle) you have to shuffle in the zig-zag manner.	
Graphs		
17	17.1 Write a Program to implement Breadth First Traversal	111
	17.2 write a program to implement Depth First Traversal	112

Reference Books:

Data Structures by Reema Thareja, Let Us C by Yashavant P. Kanetkar

EXPERIMENT-1 (Recursion)

Aim: Program to find Greatest Common Divisor for given 2 numbers.

Source code:

```
#include<stdio.h>
int gcd(int,int);
void main()
{
    int x,y;
    printf("Enter 1st number: ");
    scanf("%d",&x);
    printf("Enter 2nd number: ");
    scanf("%d",&y);
    printf("GCD of %d & %d is %d",x,y,gcd(x,y));
}
int gcd(x,y)
{
    if(y==0)
        return x;
    else
        return gcd(y,x%y);
}
```

Prelab Questions:-

- 1) What is Recursion
- 2) What is GCD
- 3) What is Prime factorization
- 4) What is Euclid's algorithm
- 5) What is Steins (Binary GCD) algorithm

Lab assignment:-

- 1) Write a program to implement GCD using Euclids and steins algorithm.

Post lab Questions:-

- 1) What are the advantages and drawbacks of prime factorization, Euclid's algorithm and steins algorithm.

Aim: Program to multiply 2 numbers without using * operator

Source code:

```
#include <stdio.h>
int product( int,int);
void main ()
```

```

{
    int a, b;

    printf("Enter two integers: ");
    scanf("%d%d",&a,&b);

    printf("\nProduct = %d", product(a, b));
}
int product(n1,n2)
{
    if(n2 == 0)
        return 0;

    if(n2 > 0)
        return (n1 + product(n1, n2 - 1));

    if(n2 < 0)
        return -product(n1, -n2);
}

```

Prelab Questions:-

- 1) What is Recursion.

Lab assignment:-

- 1) Write a program to multiplication of N numbers.
- 2) Write a program to multiply two numbers using bitwise operators.

Post lab Questions:-

- 1) Can we multiply two integers without using multiplication, division, bitwise operators and no loops

Aim: Program to divide 2 numbers without using / operator

Source code:

```

#include<stdio.h>
int rem;
int div(int,int);
void main()
{
    int n1, n2;
    printf("Enter 1st Number: ");
    scanf("%d", &n1);
    printf("Enter 2nd Number: ");
    scanf("%d", &n2);

    printf("\nDivision of %d and
%d:\nQuotient:\t%d\nRemainder:\t%d\n",n1,n2,rem,div(n1,n2));

```

```

}
int div(n1,n2)
{
    if(n1==0)
        return 0;
    else if(n1>=n2)
    {
        n1=n1-n2;
        rem++;
        return div(n1,n2);
    }
}

```

Prelab Questions:-

- 1) What is Recursion.

Lab assignment:-

- 1) Write a program to divide two numbers using bitwise operators.

Post lab Questions:-

- 1) Can we divide two integers without using division, bitwise operators and mod operator.

Aim: Program to find power of the numbers (x, y)**Source code:**

```

#include <stdio.h>
double powe(double,int);
void main()
{
    double base, power;
    int expo;
    printf("Enter base: ");
    scanf("%lf", &base);
    printf("Enter exponent: ");
    scanf("%d", &expo);

    power = powe(base, expo);

    printf("%lf ^ %d = %f", base, expo, power);
}
double powe(double base, int expo)
{
    if(expo == 0)
        return 1;
    else if(expo > 0)
        return (base * powe(base, expo-1));
    else if(expo < 0)

```



```

    return (1/powe(base,-expo));
}

```

Prelab Questions:-

- 1) What is Recursion.

Lab assignment:-

- 1) Write a program to implement the same problem for a time complexity of $O(\log n)$
- 2) Write a Power function without using multiplication and division operators.

Post lab Questions:-

- 1) Is there any algorithm that will solve the problem with less than $O(\log n)$ time

Aim: Program to Check Whether a Number is Palindrome or Not**Source code:**

```

#include<stdio.h>
int pdrme(int);
int n,rem,rev=0;
void main()
{
    int res;
    printf("Enter a Number: ");
    scanf("%d",&n);
    res=pdrme(n);
    if(res==n)
    {
        printf("Given num is Palindrome");
    }
    else
    {
        printf("Not Palindrome");
    }
}
int pdrme(n)
{
    if(n!=0)
    {
        rem=n%10;
        rev=rev*10+rem;
        pdrme(n/10);
    }
    return rev;
}

```

Prelab Questions:-

- 1) What is palindrome
- 2) Basic Palindrome program logic
- 3) What is the difference between / and %

Lab assignment:-

- 1) Write a program to check whether the String is palindrome or not
- 2) Write a program to find the minimum length of the sub-string whose characters can be used to form a palindrome of length K. If no such sub-string exists then print -1.

Post lab Questions:-

- 1) What is longest palindrome string

Aim: program for Tower of Hanoi problem**Source code:**

```
#include <stdio.h>
void toh(int,char,char,char,char);
int c=0;
void main()
{
    int n;
    printf("Enter no.of Disks : ");
    scanf("%d",&n);
    toh(n,'A','D','B','C');
    printf("\n\nTotal Moves::%d",c);
}
void toh(int n,char src,char des,char mid1,char mid2)
{
    if(n==0)
    {
        return;
    }
    if (n == 1)
    {
        printf("\n Move disk 1 from rod %c to rod %c",src,des);c++;
        return;
    }
    toh(n-2,src,mid1,mid2,des);
    printf("\n Move disk %d from rod %c to rod %c",n-1,src,mid2);c++;
    printf("\n Move disk %d from rod %c to rod %c", n,src,des); c++;
    printf("\n Move disk %d from rod %c to rod %c",n-1,mid2,des);c++;
    toh(n-2,mid1,des,src,mid2);
}
}
```

Prelab Questions:-

- 1) What is Tower of Hanoi

Lab assignment:-**1) Solve the Double tower Problem**

Statement: A Double Tower of Hanoi contains $2n$ disks of n different sizes, exactly two of each size. As before, we can move only one disk at a time and must never place a larger one onto a smaller one.

- How many steps does it take to transfer a double tower from one peg to another, if disks of equal size are indistinguishable from each other?
- What if we are required to reproduce the original order of all the equal-size disks in the final arrangement

Post lab Questions:-

- 1) How many steps are required to move total disks in Double Tower of Hanoi

EXPERIMENT-2 (Linked List)

Aim: Program to create a Single Linked List and perform all insertion and deletion cases

Source code:

```
#include<stdio.h>
#include<malloc.h>
#include<stdlib.h>
struct node{
    int dta;
    struct node *nxt;
}*nw,*tmp,*cur,*prv,*nxn,*tmp1;
struct node *st=NULL;
void ins_at_begin(int);
void ins_at_end(int);
void ins_at_spl(int);
//void traverse();
void del_frm_begin();
void del_frm_end();
void del_at_spl();
//void reverse();
int l,i=0,ct = 0,dta;
void toprint(struct node *st)
{
    while(st!=NULL)
    {
        printf("%d ",st->dta);
        st=st->nxt;
    }
}
```

```

}
void ip(){
    printf("Enter value of element : ");
    scanf("%d", &dta);
}
void main(){
    for (;;) {
        printf("\n1. Insert an element at beginning.\n");
        printf("\n2. Insert an element at end.\n");
        printf("\n3. Insert an element at specific location.\n");
        printf("\n4. Display.\n");
        printf("\n5. Delete element from beginning.\n");
        printf("\n6. Delete element from end.\n");
        printf("\n7. Delete at Specific Location.\n");
        //printf("\n8. Reverse of Linked list.\n");
        printf("\n8. Exit.\n");

        printf("Enter choice : ");
        scanf("%d", &i);

        if (i==1) {
            ip();
            ins_at_begin(dta);
        }
        else if (i==2) {
            ip();
            ins_at_end(dta);
        }
        else if (i==3) {
            ip();
            ins_at_spl(dta);
        }
        else if (i==4)
            toprint(st);
        else if (i==5)
            del_frm_begin();
        else if (i==6)
            del_frm_end();
        else if (i==7)
            del_at_spl();
        /*else if (i==8){
            toprint(st);printf("--> Linked List.\n");
            reverse();
            toprint(st);printf("--> Reverse of Linked List.\n");
        }*/
        else if (i==8){

```

```

        toprint(st);printf("--> Linked List.\n");
        break;
    }
    else{
        printf("Please enter valid input.\n");
        break;
    }
}
}
void ins_at_begin(int x){

    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        nw->data=x;
        nw->nxt=NULL;
        st=nw;
    }
    else{
        nw->data=x;
        nw->nxt=st;
        st=nw;}
}
void ins_at_spl(int x){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        st->data=x;
        st->nxt=NULL;
    }
    else {
        printf("Enter Location to insert : ");
        scanf("%d",&l);
        if(l==1)
            ins_at_begin(x);
        else{
            nw->data=x;
            tmp=st;
            for(i=2;i<=l-1;i++){
                tmp=tmp->nxt;
                if(tmp==NULL)
                    printf("Location not found.\n");
            }
            if(tmp!=NULL){
                nw->nxt=tmp->nxt;

```

```

        tmp->nxt=nw;
    }
}

void ins_at_end(int x){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        st->dta=x;
        st->nxt=NULL;
    }

    tmp=st;

    while(tmp->nxt!=NULL){
        tmp=tmp->nxt;
    }
    tmp->nxt=nw;
    nw->dta=x;
    nw->nxt=NULL;
}

/*void traverse(){
    tmp=st;
    if(tmp==NULL){
        printf("Linked List is Empty");
    }
    printf("There are %d elements in linked list.", ct);

    while(tmp->nxt!=NULL){
        printf("%d ", tmp->dta);
        tmp=tmp->nxt;
    }
    printf("%d\n",tmp->dta);
}*/

void del_frm_begin(){
    int n;
    if (st==NULL) {
        printf("Linked list is already empty.\n");
    }
    n=st->dta;
    nw=st->nxt;
    free(st);
    st=nw;
}

```

```

        ct--;
        printf("%d Deleted from beginning successfully.\n", n);
    }
    void del_frm_end(){
        int n;
        if (st==NULL)
            printf("Linked list is already empty.\n");
        ct--;
        if(st->nxt==NULL){
            n=st->dta;
            free(st);
            st=NULL;
            printf("%d Deleted from end successfully.\n", n);
        }
        nw=st;
        while(nw->nxt!=NULL){
            tmp=nw;
            nw=nw->nxt;
        }
        n=nw->dta;
        tmp->nxt=NULL;
        free(nw);
        printf("%d Deleted from end successfully.\n", n);
    }
    void del_at_spl(){
        tmp=st;
        printf("Enter loc : ");
        scanf("%d",&l);
        if(tmp==NULL)
            return;
        else if(l==1){
            del_frm_begin();
        }
        else{
            for(i=2;i<=l;i++){
                prv=tmp;
                tmp=tmp->nxt;
            }
            prv->nxt=tmp->nxt;
            free(tmp);
        }
    }
    /*void reverse(){
        cur=st;
        prv=NULL;

```

```

    nxn=NULL;
    while(cur!=NULL){
        nxn=cur->nxt;
        cur->nxt=prv;
        prv=cur;
        cur=nxn;
    }
    st=prv;
}*/

```

Prelab Questions:-

1. What is data structure
2. How the memory is allocated dynamically
3. What is linked list
4. What is node
5. What are the types of linked list

Lab assignment:-

- 1) Write a program to insert a node after the nth node from right end position.
- 2) Write a program to delete all duplicate elements in SLL.

Post lab Questions:-

- 1) What are the advantages and disadvantages of Linked list
- 2) How to traverse a linked list in reverse order

Aim: Program to create a Circular Linked List and perform all insertion and deletion cases

Source code:

```

#include<stdio.h>
#include<malloc.h>
#include<stdlib.h>
struct node{
    int dta;
    struct node *nxt;
}*nw,*tmp,*cur,*prv,*nxn,*tmp1;
struct node *st=NULL;
void ins_at_begin();
void ins_at_end();
void ins_at_spl();
//void traverse();
void del_frm_begin();
void del_frm_end();
void del_at_spl();
//void reverse();
int l,i=0,ct = 0,dta,p=0,n;

```



```

void toprint()
{ tmp=st;
  while(tmp->nxt!=st)
  {
    printf("%d ",tmp->dta);
    tmp=tmp->nxt;
  }
  printf("%d ",tmp->dta);
}
void ip(){
  printf("Enter value of element : ");
  scanf("%d", &nw->dta);
}
void main(){
  for (;;) {
    printf("\n1. Insert an element at beginning.\n");
    printf("2. Insert an element at end.\n");
    printf("3. Insert an element at Specific Location.\n");
    printf("4. Display.\n");
    printf("5. Delete element from beginning.\n");
    printf("6. Delete element from end.\n");
    printf("7. Delete at Specific Location.\n");
    //printf("8. Reverse of Linked list.\n");
    printf("8. Exit.\n");

    printf("Enter choice : ");
    scanf("%d", &i);

    if (i==1) {
      ins_at_begin();
      ip();
    }
    else if (i==2) {
      ins_at_end();
      ip();
    }
    else if (i==3) {
      ins_at_spl();
      if(p!=0)
        printf("Location not found.\n");
      else
        ip();
    }
    else if (i==4)
      toprint();
    else if (i==5){
      del_frm_begin();
      printf("%d Deleted from end successfully.\n", n);continue;}
    else if (i==6){

```

```

del_frm_end();
printf("%d Deleted from end successfully.\n", n);continue;}
else if(i==7){
    del_at_spl();
    printf("%d Deleted from end successfully.\n", n);continue;}
/*else if (i==8){
    toprint();printf("--> Linked List.\n");
    reverse();
    toprint();printf("--> Reverse of Linked List.\n");
}*/
else if(i==8){
    if(ct==1 || ct==0)
        printf("NULL\n");
    else{
        toprint();
        printf("--> Linked List.\n");}
    break;
}
else{
    printf("Please enter valid input.\n");
    break;
}
}
}
void ins_at_begin(){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        nw->nxt=st;
    }
    else{
        tmp=st;
        while(tmp->nxt!=st)
            tmp=tmp->nxt;
        tmp->nxt=nw;
        nw->nxt=st;
        st=nw;
    }
}
void ins_at_spl(){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        nw->nxt=st;
    }
    else {
        printf("Enter Location to insert : ");

```

```

        scanf("%d",&l);
        if(l==1)
            ins_at_begin();
        else{
            tmp=st;
            for(i=2;i<=l-1;i++){
                tmp=tmp->nxt;
                if(tmp==st){
                    p++;
                    return;} }
            if(tmp->nxt!=st){
                nw->nxt=tmp->nxt;
                tmp->nxt=nw;}
            else if(tmp->nxt==st)
                ins_at_end();
        }
    }
}

void ins_at_end(){

    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        nw->nxt=st;
    }
    else{
        tmp=st;
        while(tmp->nxt!=st){
            tmp=tmp->nxt;
        }
        tmp->nxt=nw;
        nw->nxt=st;
    }
}

/*void traverse(){
    tmp=st;
    if(st==NULL){
        printf("Linked List is Empty");
    }
    printf("There are %d elements in linked list.", ct);

    while(tmp->nxt!=st){
        printf("%d ", tmp->dta);
        tmp=tmp->nxt;
    }
    printf("%d\n",tmp->dta);
}

```

```

*/
void del_frm_begin(){
    tmp=st;
    tmp1=st;
    n=st->dta;
    if (st==NULL)
        printf("Linked list is already empty.\n");
    else{
        while(tmp->nxt!=st)
            tmp=tmp->nxt;
        tmp->nxt=tmp1->nxt;
        st=tmp1->nxt;
        free(tmp1);}

    ct--;
}

void del_frm_end(){
    tmp=st;
    tmp1=st;
    if (st==NULL)
        printf("Linked list is already empty.\n");
    else{
        while(tmp->nxt!=st){
            tmp1=tmp;
            tmp=tmp->nxt; }

        }
    n=tmp->dta;
    tmp1->nxt=st;
    free(tmp);
    ct--;
}

void del_at_spl(){
    tmp=st;
    printf("Enter loc : ");
    scanf("%d",&l);
    if(st==NULL)
        printf("Already Empty.\n");
    else{
        for(i=1;i<=l;i++){
            tmp1=tmp;
            n=tmp->dta;
            tmp=tmp->nxt;

        }
        tmp1->nxt=tmp->nxt;
        free(tmp);
        ct--;
    }
}

```

```

/*void reverse(){
    tmp=st;
    prv=st;
    cur=st->nxt;
    st=st->nxt;
    while(st!=tmp){
        st=st->nxt;
        cur->nxt=prv;
        prv=cur;
        cur=st;
    }
    cur->nxt=prv;
    st=prv;
}*/

```

Prelab Questions:-

- 1) What is circular linked list
- 2) What is the problem of Single linked list

Lab assignment:-

- 1) Write a program to insert a node after the nth node from right end position.
- 2) Write a program to delete all duplicate elements in CLL.
- 3) Write a program to Sort & reverse the data after nth position in circular Linked List.
- 4) There are n persons who are arranged in circular passion order (CLL). Find the winner after deleting k-th element respectively.

Post lab Questions:-

- 1) What are the advantages and disadvantages of circular Linked list
- 2) How to traverse a circular linked list.

Aim: Program to create a Double Linked List and perform all insertion and deletion cases

Source code:

```

#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node *nxt,*prv;
}*nw,*tmp,*cur,*pst,*nxn,*tmp1;
struct node *st=NULL;
void ins_at_begin(int);
void ins_at_end(int);
void ins_at_spl(int);

```

```

//void traverse();
void del_frm_begin();
void del_frm_end();
void del_at_spl();
//void reverse();
int l,i=0,ct = 0,dta,n;
void toprint()
{ tmp=st;
  if(tmp==NULL)
    printf("Empty!!");
  while(tmp->nxt!=NULL)
  {
    printf("%d ",tmp->dta);
    tmp=tmp->nxt;
  }
  printf("%d ",tmp->dta);
}
void ip(){
  printf("Enter value of element : ");
  scanf("%d", &dta);
}
void main(){
  for (;;) {
    printf("\n1. Insert an element at beginning.\n");
    printf("2. Insert an element at end.\n");
    printf("3. Insert an element at Secific Location.\n");
    printf("4. Display.\n");
    printf("5. Delete element from beginning.\n");
    printf("6. Delete element from end.\n");
    printf("7. Delete at Specific Location.\n");
    //printf("8. Reverse of Linked ist.\n");
    printf("8. Exit.\n");

    printf("Enter choice : ");
    scanf("%d", &i);

    if (i==1) {
      ip();
      ins_at_begin(dta);
    }
    else if (i==2) {
      ip();
      ins_at_end(dta);
    }
    else if (i==3) {
      ip();

```

```

        ins_at_spl(dta);
    }
    else if (i==4)
        toprint();
    else if (i==5)
        del_frm_begin();
    else if (i==6)
        del_frm_end();
    else if (i==7)
        del_at_spl();
    /* else if (i==8){
        toprint(st);printf("--> Linked List.\n");
        reverse();
        toprint(st);printf("--> Reverse of Linked List.\n");
    }*/
    else if (i==8){
        toprint();printf("--> Linked List.\n");
        break;
    }
    else{
        printf("Please enter valid input.\n");
        break;
    }
}
}
void ins_at_begin(int x){

    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    if(st==NULL){
        st=nw;
        st->dta=x;
        st->prv=NULL;
        st->nxt=NULL;
    }
    else{
        nw->dta=x;
        nw->nxt=st;
        nw->prv=NULL;
        st->prv=nw;
        st=nw;
    }
}
void ins_at_spl(int x){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;

```

```

    if(st==NULL){
        st=nw;
        st->dta=x;
        st->prv=NULL;
        st->nxt=NULL;
    }
    else {
        printf("Enter Location to insert : ");
        scanf("%d",&l);
        if(l==1)
            ins_at_begin(x);
        else{
            nw->dta=x;
            tmp=st;
            for(i=2;i<=l-1;i++){
                tmp=tmp->nxt;
                if(tmp==NULL)
                    printf("*-----Location not found-----*\n");
            }
            if(tmp!=NULL){
                nw->nxt=tmp->nxt;
                nw->prv=tmp;
                if(tmp->nxt!=NULL)
                    tmp->nxt->prv=nw;
                tmp->nxt=nw;
            }
        }
    }
}

void ins_at_end(int x){
    nw=(struct node*)malloc(sizeof(struct node));
    ct++;
    nw->dta=x;
    if(st==NULL){
        st=nw;
        st->dta=x;
        st->nxt=NULL;}
    tmp=st;
    while(tmp->nxt!=NULL){
        tmp=tmp->nxt;}
    tmp->nxt=nw;
    nw->prv=tmp;
    nw->nxt=NULL;
    tmp=nw;
}

/*void traverse(){

```



```

tmp=st;

if(tmp==NULL){
    printf("Linked List is Empty");
}
printf("There are %d elements in linked list.", ct);

while(tmp->nxt!=NULL){
    printf("%d ", tmp->dta);
    tmp=tmp->nxt;
}
printf("%d\n", tmp->dta);
}*/
void del_frm_begin(){
    if (st==NULL) {
        printf("Linked list is already empty.\n");
    }
    n=st->dta;
    tmp=st;
    st=st->nxt;
    if(st!=NULL)
        st->prv=NULL;
    free(tmp);
    ct--;
    printf("*--%d Deleted from beginning successfully--*\n", n);
}
void del_frm_end(){
    if (st == NULL)
        printf("Linked list is already empty.\n");
    ct--;
    if(st->nxt==NULL){
        n=st->dta;
        free(st);
        st=NULL;
        printf("%d Deleted from end successfully.\n", n);
    }
    tmp=st;
    while(tmp->nxt!=NULL){
        tmp1=tmp;
        tmp=tmp->nxt;
    }
    n=tmp->dta;
    tmp1->nxt=NULL;
    free(tmp);
    printf("%d Deleted from end successfully.\n", n);
}
}

```

```

void del_at_spl(){
    tmp=st;
    printf("Enter loc : ");
    scanf("%d",&l);
    if(tmp==NULL)
        printf("Linked list is already empty.\n");
    else if(l==1){
        del_frm_begin();
    }
    else{
        for(i=2;i<=l;i++){
            pst=tmp;
            tmp=tmp->nxt;}
        }
        pst->nxt=tmp->nxt;
        tmp->prv=pst;
        free(tmp);
    }
}
/*void reverse(){
    cur=st;
    tmp=NULL;
    while(cur!=NULL){
        tmp=cur->prv;
        cur->prv=cur->nxt;
        cur->nxt=tmp;
        cur=cur->prv;
    }
    st=tmp->prv;
}*/

```

Prelab Questions:-

- 1) What is Double linked list
- 2) What is the problem of single or circular linked list
- 3) How to represent a node in double linked list.

Lab assignment:-

- 1) Write a program to accept a value from the user, if the value exists: Exchange the previous node value and after node value along with display otherwise display no data found.
- 2) Write a program to Accept a value from the user, if the value exists display it's previous node and after node value otherwise display no data found.

Post lab Questions:-

- 1) What are the differences between single and double linked list
- 2) How to traverse in double linked list.

Aim: Program to create a single Linked List and reverse all the links.

Source code:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int dta;
    struct node *nxt;
}*nw,*tmp,*cur,*prv,*nxn;
struct node *st=NULL;
void ins_at_end(int);
void reverse();
int i=0,dta;
void toprint(struct node *st)
{
    while(st!=NULL)
    {
        printf("%d ",st->dta);
        st=st->nxt;
    }
}
void ip(){
    printf("Enter value of element : ");
    scanf("%d", &dta);
}
void main(){
    for (;;) {
        printf("\n1. Insert.\n");
        printf("2. Display Linked List.\n");
        printf("3. Reverse of Linked list.\n");
        printf("4. Exit.\n");

        printf("Enter choice : ");
        scanf("%d", &i);

        if (i==1) {
            ip();
            ins_at_end(dta);
        }
        else if (i==2)
            toprint(st);
        else if (i==3){
            toprint(st);printf("--> Linked List.\n");
            reverse();
            toprint(st);printf("--> Reverse of Linked List.\n");
        }
    }
}
```

```

        else if(i==4)
            break;
    else{
        printf("Please enter valid input.\n");
        break;
    }
}
}
void ins_at_end(int x){

    nw=(struct node*)malloc(sizeof(struct node));
    if(st==NULL){
        st=nw;
        st->dta=x;
        st->nxt=NULL;
    }

    tmp=st;

    while(tmp->nxt!=NULL){
        tmp=tmp->nxt;
    }
    tmp->nxt=nw;
    nw->dta=x;
    nw->nxt=NULL;
}
void reverse(){
    cur=st;
    prv=NULL;
    nxn=NULL;
    while(cur!=NULL){
        nxn=cur->nxt;
        cur->nxt=prv;
        prv=cur;
        cur=nxn;
    }
    st=prv;
}
}

```

Prelab Questions:-

- 1) What are the different ways to sort the data in Linked list

Lab assignment:-

- 1) Write a program to Sort & reverse the data after nth position in Single Linked List.
- 2) Write a program to sort the data (Not the links)
- 3) Write a program to implement the following operations in Single linked list with appropriate messages.
 - i) Insert details of students like ID, name and one subject marks

ii) Display the Student details in sorting order of marks

Post lab Questions:-

- 1) What are the different types of sorting algorithms, mention it's advantages and disadvantages?

Aim: Program to store polynomial in a linked list and apply add and subtract operations on two polynomials

Source code:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int pw;
    float cf;
    struct node* nxt;
}*tmp,*prv,*cur,*nw,*w,*x,*y,*z;
struct node* insert(struct node* st,int c,int p);
struct node* append(struct node* st,int c,int p);
struct node* polyadd(struct node* p1,struct node* p2);
struct node* polysub(struct node* p1,struct node* p2);
void dsply(struct node* st);
int main()
{
    int a,b,n,i,j;
    for(i=1;i<=2;i++){
        printf("\nEnter the no of terms of polynomial %d : ",i);
        scanf("%d",&n);
        for(j=0;j<n;j++){
            printf("\nEnter the coefficient and exponent of the term : ");
            scanf("%d%d",&a,&b);
            if(i==1)
                w=insert(w,a,b);
            else if(i==2)
                x=insert(x,a,b);
        }
    }
    printf("\nThe polynomial 1 : ");
    dsply(w);
    printf("\n\nThe polynomial 2 : ");
    dsply(x);
    y=polyadd(w,x);
    printf("\n\nThe Sum of two polynomials : ");
    dsply(y);
    z=polysub(w,x);
    //Perform Subtraction.
    //Displaying the polynomials..
    //Performing Addition.
```

```

printf("\n\nThe Sub of two polynomials : ");
dsply(z);
}
struct node* append(struct node* st,int c,int p){
    nw = (struct node*)malloc(sizeof(struct node));
    nw->cf=c;
    nw->pw=p;
    if(st==NULL){
        nw->nxt=NULL;
        return nw;
    }
    tmp=st;
    while(tmp->nxt!=NULL)
        tmp=tmp->nxt;
    tmp->nxt=nw;
    nw->nxt=NULL;
    return st;
}
struct node* insert(struct node* st,int c,int p){
    nw=(struct node*)malloc(sizeof(struct node));
    nw->cf=c;
    nw->pw=p;
    if(st==NULL){
        nw->nxt=NULL;
        return nw;
    }
    prv=cur=st;
    while(cur!=NULL && cur->pw>p){
        prv=cur;
        cur=cur->nxt;
    }
    if(cur==st){
        nw->nxt=cur;
        return nw;
    }
    else if(cur==NULL){
        prv->nxt=nw;
        nw->nxt=NULL;
    }
    else{
        nw->nxt=cur;
        prv->nxt=nw;
    }
    return st;
}
struct node* polyadd(struct node* p1,struct node* p2){
    struct node* ans=NULL;

```

// Corner Case to handle if the list is empty...

// Traversing to point to the last node...

// for inserting the first node..

// for inserting before the first node...

//for inserting after the last node....

```

struct node* t1,* t2;
t1=p1;
t2=p2;
while(t1!=NULL && t2!=NULL){
    if(t1->pw > t2->pw){
        ans=append(ans,t1->cf,t1->pw);
        t1=t1->nxt;
    }
    else if(t1->pw < t2->pw){
        ans=append(ans,t2->cf,t2->pw);
        t2=t2->nxt;
    }
    else{
        ans=append(ans,(t1->cf)+(t2->cf),t1->pw);
        t1=t1->nxt;
        t2=t2->nxt;
    }
}
while(t1!=NULL){          //coping the remaining terms of polynomial 1...
    ans=append(ans,t1->cf,t1->pw);
    t1=t1->nxt;
}
while(t2!=NULL){          //coping the remaining terms of polynomial 2...
    ans=append(ans,t2->cf,t2->pw);
    t2=t2->nxt;
}
return ans;
}

struct node* polysub(struct node* p1,struct node* p2){
    struct node* ans=NULL;
    struct node* t1,* t2;
    t1=p1;
    t2=p2;
    while(t1!=NULL && t2!=NULL){
        if(t1->pw > t2->pw){
            ans=append(ans,t1->cf,t1->pw);
            t1=t1->nxt;
        }
        else if(t1->pw < t2->pw){
            ans=append(ans,t2->cf,t2->pw);
            t2=t2->nxt;
        }
        else{
            ans=append(ans,(t1->cf)-(t2->cf),t1->pw);
            t1=t1->nxt;
            t2=t2->nxt;
        }
    }
}

```

```

    }
    while(t1!=NULL){        //coping the remaining terms of polynomial 1...
        ans=append(ans,t1->cf,t1->pw);
        t1=t1->nxt;
    }
    while(t2!=NULL){        //coping the remaining terms of polynomial 2...
        ans=append(ans,t2->cf,t2->pw);
        t2=t2->nxt;
    }
    return ans;
}

void dsply(struct node* st){
    tmp=st;
    if(tmp==NULL)
        printf("\nEmpty..");
    else{
        while(tmp->nxt!=NULL){
            if(tmp->cf!=0){
                printf(" (%.1f)x^%d ",tmp->cf,tmp->pw);
                if(tmp->nxt->cf!=0)
                    printf("+");
            }
            tmp=tmp->nxt;
        }
        if(tmp->cf!=0)
            printf(" (%.1f)x^%d ",tmp->cf,tmp->pw);
    }
}

```

Prelab Questions:-

- 1) What is a polynomial.
- 2) How to represent a polynomial function in linked list

Lab assignment:-

- 1) Write a program for multiplying the given two polynomial functions.

Post lab Questions:-

- 2) Can we display the result of multiplication of two polynomials without using third linked list

Aim: Program to sort the numbers given in a Linked List**Source code:**

```

#include <stdio.h>
#include <stdlib.h>
struct node{
    int dta;
    struct node *nxt;
}*nw,*tmp,*tmp1,*tmp2;

```



```

struct node *st=NULL;
int i,n,x,c;
void toprint(struct node *st)
{
    while(st!=NULL)
    {
        printf("%d ",st->dta);
        st=st->nxt;
    }
}
void ascending(){
    tmp1=st;
    while(tmp1->nxt!=NULL){
        tmp2=st;
        while(tmp2->nxt!=NULL){
            if((tmp2->dta)>(tmp2->nxt->dta)){
                x=tmp2->dta;
                tmp2->dta=tmp2->nxt->dta;
                tmp2->nxt->dta=x;
            }
            tmp2=tmp2->nxt;
        }
        tmp1=tmp1->nxt;
    }
    toprint(st);
}
void descending(){
    tmp1=st;
    while(tmp1->nxt!=NULL){
        tmp2=st;
        while(tmp2->nxt!=NULL){
            if((tmp2->dta)<(tmp2->nxt->dta)){
                x=tmp2->dta;
                tmp2->dta=tmp2->nxt->dta;
                tmp2->nxt->dta=x;
            }
            tmp2=tmp2->nxt;
        }
        tmp1=tmp1->nxt;
    }
    toprint(st);
}
void main(){
    printf("Enter no.of elements to create : ");
    scanf("%d",&n);
    for(i=1;i<=n;i++){
        nw=(struct node*)malloc(sizeof(struct node));

```

```

printf("Enter %d node data : ",i);
scanf("%d",&nw->dta);
if(st==NULL){
    st=nw;
    st->nxt = NULL;}
else{
    tmp=st;
    while(tmp->nxt!=NULL){
        tmp=tmp->nxt;}
    tmp->nxt=nw;
    nw->nxt=NULL;}
}
toprint(st);printf("\n");
printf("-- Select type of sorting --\n1.Ascending\n2.Descending\n");
printf("Enter choice : ");
scanf("%d",&c);
if(c==1)
    ascending();
else if(c==2)
    descending();
else
    printf("***Invalid Choice***");
}

```

Prelab Questions:-

- 1) What are the different ways to sort the data in Linked list

Lab assignment:-

- 1) Write a program to display ith position element after placed in sorting order i.e., Ordered statistics

Post lab Questions:-

- 1) Can we apply binary search algorithm in linked list.

Aim: Program to create LL which stores details of students and print the name of the students who got first class

Source code:

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
struct node{
    int rnk;
    char dta[100];
    struct node *nxt;
}*nw,*tmp;
struct node *st=NULL;

```

```

void ins(int, char dta[100]);
void toprint(struct node*);
char dta[100];
int i=0, rnk;
void ip(){
    printf("Enter name of student : ");
    scanf("%s", dta);
    printf("Enter rank of student : ");
    scanf("%d", &rnk);
}
void main(){
    for (;;) {
        printf("\n1. Insert.\n");
        printf("2. Display Linked List.\n");
        printf("3. Exit.\n");

        printf("Enter choice : ");
        scanf("%d", &i);

        if (i==1) {
            ip();
            ins(rnk, dta);
        }
        else if (i==2)
            toprint(st);
        else if (i==3)
            break;
        else{
            printf("Please enter valid input.\n");
            break;
        }
    }
}

void ins(int y, char dta[100]){
    nw=(struct node*)malloc(sizeof(struct node));
    if(st==NULL){
        st=nw;
        strcpy(st->dta, dta);
        st->rnk=y;
        st->nxt=NULL;
    }

    tmp=st;

    while(tmp->nxt!=NULL){
        tmp=tmp->nxt;
    }
}

```

```

    }
    tmp->nxt=nw;
    strcpy(nw->dta,dta);
    nw->rnk=y;
    nw->nxt=NULL;
}
void toprint(struct node *st)
{
    printf("-----\n");
    printf("|Name\t\tRank \t|\n");
    while(st!=NULL)
    {
        printf("\n|s\t\t%d\t|",st->dta,st->rnk);
        if(st->rnk==1)
            printf("Topper is %s",st->dta);
        st=st->nxt;
    }
    printf("\n-----");
}

```

Prelab Questions:-

- 1) How to compare two strings

Lab assignment:-

- 1) Write a LL program for read string from user then u have to ask for enter a sub-sequence string then you have to display whether it is a sub-sequence string or not.

Post lab Questions:-

- 1) Can we store more than two fields in a node.

EXPERIMENT-3 (Stacks)

Aim: Program to perform push, pop and peek operations on a stack using arrays

Source code:

```

#include<stdio.h>
#define MAX 10000

int top=-1,i,x,stack[MAX];
void push();
void pop();
void peek();
void display();

```

```

void main()
{
    int ch,c=0;
    while(!c)
    {
        printf("1.Push\n2.Pop\n3.Display\n4.Peek\n5.Exit\n");
        printf("Enter your choice : ");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1: push();
                    break;
            case 2: pop();
                    break;
            case 3: display();
                    break;
            case 4: peek();
                    break;
            case 5: c++;break;
            default: printf("\nWrong Choice!!");
        }
    }
}

void push()
{
    if(top==MAX-1)
        printf("\nStack is full!!");
    else
    {
        printf("Enter element to push:");
        scanf("%d",&x);
        top=top+1;
        stack[top]=x;
    }
}

void pop(){
    if(top==-1)
        printf("\nStack is empty!!");
    else
    {
        printf("\nDeleted element is %d\n",stack[top]);
        top=top-1;
    }
}

void peek(){
    if(top==-1)

```

```

        printf("\nStack is empty!!");
    else
        printf("%d\n",stack[top]);
}
void display()
{
    if(top==-1)
    {
        printf("\nStack is empty!!");
    }
    else
    {
        printf("\nStack is...\n");
        for(i=top;i>=0;--i)
            printf("%d\n",stack[i]);
    }
}

```

Prelab Questions:-

- 1) What is stack
- 2) What are the different operations performed on the stack
- 3) What are the real time examples of stack

Lab assignment:-

- 1) Write a program to reverse the data in stack using arrays

Post lab Questions:-

- 1) What are the applications of stack
- 2) Where do we mainly use stack in the computer.

Aim: Program to implement stack using Linked List**Source code:**

```

#include<stdio.h>
#include<stdlib.h>

struct Node
{
    int data;
    struct Node *next;
}*top = NULL;

void push(int);
void pop();
void display();

void main()

```

```

{
    int choice, value, c=0;
    while(!c){
        printf("1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&choice);
        switch(choice){
            case 1: printf("Enter the value to be insert: ");
                    scanf("%d", &value);
                    push(value);
                    break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: c++;break;
            default: printf("\nWrong selection!!! Please try again!!!\n");
        }
    }
}

void push(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    if(top == NULL)
        newNode->next = NULL;
    else
        newNode->next = top;
    top = newNode;
    printf("\nInsertion is Success!!!\n");
}

void pop()
{
    if(top == NULL)
        printf("\nStack is Empty!!!\n");
    else{
        struct Node *temp = top;
        printf("\nDeleted element: %d\n", temp->data);
        top = temp->next;
        free(temp);
    }
}

void display()
{
    if(top == NULL)
        printf("\nStack is Empty!!!\n");
    else{

```

```

struct Node *temp = top;
while(temp->next != NULL){
    printf("%d ",temp->data);
    temp = temp -> next;
}
printf("%d\n",temp->data);
}
}

```

Prelab Questions:-

- 1) What is stack
- 2) What are the different operations performed on the stack
- 3) What are the real time examples of stack

Lab assignment:-

- 1) Write a program to reverse the data in stack using linked list

Post lab Questions:-

- 1) What are the applications of stack
- 2) Where do we mainly use stack in the computer.

Aim: Program to reverse the contents of stack**Source code:**

```

#include <stdio.h>
#include <stdlib.h>
struct node{
    int a;
    struct node *next;
};
void generate(struct node **);
void display(struct node *);
void stack_reverse(struct node **);
void del(struct node **);
void main(){
    struct node *head = NULL;

    generate(&head);
    printf("\nThe sequence of contents in stack\n");
    display(head);
    printf("\nInversing the contents of the stack\n");
    stack_reverse(&head);
    printf("\nThe contents in stack after reversal\n");
    display(head);
    del(&head);
}
void stack_reverse(struct node **head){
    struct node *temp, *prev;

```



```

if (*head == NULL)
{
    printf("Stack does not exist\n");
}
else if ((*head)->next == NULL)
{
    printf("Single node stack reversal brings no difference\n");
}
else if ((*head)->next->next == NULL)
{
    (*head)->next->next = *head;
    *head = (*head)->next;
    (*head)->next->next = NULL;
}
else
{
    prev = *head;
    temp = (*head)->next;
    *head = (*head)->next->next;
    prev->next = NULL;
    while ((*head)->next != NULL)
    {
        temp->next = prev;
        prev = temp;
        temp = *head;
        *head = (*head)->next;
    }
    temp->next = prev;
    (*head)->next = temp;
}
}

void display(struct node *head){
    if (head != NULL)
    {
        printf("%d ", head->a);
        display(head->next);
    }
}

void generate(struct node **head){
    int num, i, dta;
    struct node *temp;

    printf("Enter length of stack: ");
    scanf("%d", &num);
    for (i = num; i > 0; i--)

```

```

{
    printf("Enter %d element : ",i);
    scanf("%d",&data);
    temp = (struct node *)malloc(sizeof(struct node));
    temp->a = data;
    if (*head == NULL)
    {
        *head = temp;
        (*head)->next = NULL;
    }
    else
    {
        temp->next = *head;
        *head = temp;
    }
}
}
void del(struct node **head)
{
    struct node *temp;
    while (*head != NULL)
    {
        temp = *head;
        *head = (*head)->next;
        free(temp);
    }
}

```

Prelab Questions:-

- 1) What is the general procedure to reverse the data

Lab assignment:-

- 1) Write a program to count total number of times a particular character or number appeared.

Post lab Questions:-

- 1) How many values can be popped at a time from stack.

Aim: Program to check nesting of parenthesis**Source code:**

```

#include<stdio.h>
#include<string.h>
#define MAX 1000
int top=-1;
int stack[MAX];

void push(char);

```

```

char pop();
int match(char a,char b);
void main()
{
    char exp[MAX];
    int valid;
    printf("Enter an algebraic expression : ");
    gets(exp);
    valid=check(exp);
    if(valid==1)
        printf("Valid expression\n");
    else
        printf("Invalid expression\n");
}
int check(char exp[] )
{
    int i;
    char temp;
    for(i=0;i<strlen(exp);i++)
    {
        if(exp[i]=='(' || exp[i]=='{' || exp[i]=='[')
            push(exp[i]);
        if(exp[i]==')' || exp[i]=='}' || exp[i]==']')
            if(top==-1) /*stack empty*/
            {
                printf("Right parentheses are more than left parentheses\n");
                return 0;
            }
        else
        {
            temp=pop();
            if(!match(temp, exp[i]))
            {
                printf("Mismatched parentheses are : ");
                printf("%c and %c\n",temp,exp[i]);
                return 0;
            }
        }
    }
    if(top==-1) /*stack empty*/
    {
        printf("Balanced Parentheses\n");
        return 1;
    }
    else
    {

```

```

        printf("Left parentheses more than right parentheses\n");
        return 0;
    }
}/*End of main()*/
int match(char a,char b)
{
    if(a=='[' && b==']')
        return 1;
    if(a=='{' && b=='}')
        return 1;
    if(a=='(' && b==')')
        return 1;
    return 0;
}/*End of match()*/
void push(char item)
{
    if(top==(MAX-1))
    {
        printf("Stack Overflow\n");
        return;
    }
    top=top+1;
    stack[top]=item;
}/*End of push()*/
char pop()
{
    if(top==-1)
    {
        printf("Stack Underflow\n");
    }
    return(stack[top--]);
}/*End of pop()*/

```

Prelab Questions:-

- 1) What is valid and invalid expression

Lab assignment:-

- 1) Write a program to count total number times exactly every type of parenthesis matched.

Post lab Questions:-

- 1) Is (a+(b-C)) a valid expression
- 2) Is (a+{b-c}) a valid expression

Aim: Program to convert infix to postfix and prefix**Source code:**

```

#include <stdio.h>
#include <string.h>
#define MAX 20
int top=-1;
char stack[MAX];
//-----Prefix-----
void infix_prefix(char infix[20],char prefix[20]);
void reverse(char array[30]);
char pop();
void push(char);
int isOperator(char);
int prcd(char);
void infix_prefix(char infix[20],char prefix[20]) {
    int i,j=0;
    char symbol;
    stack[++top]='#';
    reverse(infix);
    for (i=0;i<strlen(infix);i++) {
        symbol=infix[i];
        if (isOperator(symbol)==0) {
            prefix[j]=symbol;
            j++;
        } else {
            if (symbol=='(') {
                push(symbol);
            } else if(symbol == '(') {
                while (stack[top]!='(') {
                    prefix[j]=pop();
                    j++;
                }
                pop();
            } else {
                if (prcd(stack[top])<=prcd(symbol)) {
                    push(symbol);
                } else {
                    while(prcd(stack[top])>=prcd(symbol)) {
                        prefix[j]=pop();
                        j++;
                    }
                    push(symbol);
                }
            }
        }
    }
    //end for else
}

```

```

        }
        //end for else
    }
    //end for for
    while (stack[top]!='#') {
        prefix[j]=pop();
        j++;
    }
    prefix[j]='\0';
}

void reverse(char array[30]) // for reverse of the given expression
{
    int i,j;
    char temp[100];
    for (i=strlen(array)-1,j=0;i+1!=0;--i,++j) {
        temp[j]=array[i];
    }
    temp[j]='\0';
    strcpy(array,temp);
    //return array;
}

char pop() {
    char a;
    a=stack[top];
    top--;
    return a;
}

void push(char symbol) {
    top++;
    stack[top]=symbol;
}

int pred(char symbol) // returns the value that helps in the precedence
{
    switch(symbol) {
        case '+':
        case '-':
            return 2;
        break;
        case '*':
        case '/':
            return 4;
        break;
        case '$':
        case '^':
            return 6;
        break;
    }
}

```

```

        case '#':
        case '(':
        case ')':
            return 1;
        break;
    }
}

int isOperator(char symbol) {
    switch(symbol) {
        case '+':
        case '-':
        case '*':
        case '/':
        case '^':
        case '$':
        case '&':
        case '(':
        case ')':
            return 1;
        break;
        default:
            return 0;
    }
}

//-----Postfix-----
void infix_postfix(char infix[20]);
ppush(char elem)
{
    /* Function for PUSH operation */
    stack[++top]=elem;
}

char ppop()
{
    /* Function for POP operation */
    return(stack[top--]);
}

int pr(char elem)
{
    /* Function for precedence */
    switch(elem)
    {
        case '#': return 0;
        case '(': return 1;
        case '+':
        case '-': return 2;
        case '*':
        case '/': return 3;
    }
}

```

```

    }
}
void infix_postfix(char infix[20]){
    char postfix[20],ch,elem;
    int i=0,k=0;
    ppush('#');
    while( (ch=infix[i++]) != '\0'){
        if( ch == '(')
            ppush(ch);
        else{
            if(isalnum(ch))
                postfix[k++]=ch;
            else{
                if( ch == ')'){
                    while( stack[top] != '(')
                        postfix[k++]=ppop();
                    elem=ppop();
                }
                else{
                    while( pr(stack[top]) >= pr(ch) )
                        postfix[k++]=ppop();
                    ppush(ch);
                }
            }
        }
    }
    while( stack[top] != '#')
        postfix[k++]=ppop();
    postfix[k]='\0';
    printf("%s",postfix);
}
//-----Main-----
void main(){
    char infix[20],prefix[20],temp;
    int c;
    printf("Enter infix operation: ");
    scanf("%s",infix);
    printf("\n1.Postfix :t");
    infix_postfix(infix);
    printf("\n2.Prefix :t");
    infix_prefix(infix,prefix);
    reverse(prefix);
    puts((prefix));
}

```


Prelab Questions:-

- 1) What is an expression
- 2) What are infix , prefix and postfix notations
- 3) Which data structure is used for to convert infix to postfix or prefix

Lab assignment:-

- 1) Convert the expression $(A+B) / (C + D) - (D * E)$ into post fix and prefix
- 2) Convert the expression $A-(B/C+(D*E*F)/G)*H$ into postfix and prefix

Post lab Questions:-

- 1) What are polish and reverse polish notations.

Aim: Program to evaluate a postfix and prefix expressions**Source code:**

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#define MAX 30
#define OPERAND 10
#define OPERATOR 20

typedef struct prexp
{
    int top;
    int stack[MAX];
}stck;

void init(stck*);
void push(stck *,int);
int pop(stck *);
void eval(stck *,char,int,int);
int gettype(char);
void postfix(char pos[MAX]);
void prefix(char pre[MAX]);
void main()
{
    char pos[MAX],pre[MAX];
    postfix(pos);
    prefix(pre);
}

void postfix(char pos[MAX]){
    int num1,num2,item,l,i,pr;
    stck stk;
    init(&stk);
    printf("Enter postfix expression : ");
    printf("Evaluation of postfix expression : ",gets(pos));
    for(i=0;pos[i]!='\0';i++)
```

```

    {
        if(pos[i]==' ' || pos[i]=='\t')
            continue;

        switch(gettype(pos[i]))
        {
            case OPERAND: item=pos[i]-'0';
                push(&stk,item);
                break;

            case OPERATOR: num1=pop(&stk);
                num2=pop(&stk);
                eval(&stk,pos[i],num2,num1);
        }
    }
    printf("%d\n",stk.stack[0]);
}

void prefix(char pre[MAX]){
    int num1,num2,item,l,i,pr;
    stck stk;
    init(&stk);
    printf("Enter prefix expression : ");
    printf("Evaluation of prefix expression : ",gets(pre));
    l=strlen(pre);
    for(i=l-1;i>=0;i--){
        if(pre[i]==' ' || pre[i]=='\0')
            continue;
        switch(gettype(pre[i]))
        {
            case OPERAND : item=pre[i]-'0';
                push(&stk,item);
                break;
            case OPERATOR : num1=pop(&stk);
                num2=pop(&stk);
                eval(&stk,pre[i],num1,num2);
        }
    }
    printf("%d",stk.stack[0]);
}

void init(stck *st)
{
    st->top=-1;
}

void push(stck *st,int num)
{
    st->top++;
}

```

```

    st->stack[st->top]=num;
}

int pop(stck *st)
{
    int num;
    num=st->stack[st->top];
    st->top--;
    return num;
}

void eval(stck *st,char op,int num1,int num2)
{
    int res;
    switch(op)
    {
        case '+': res=num1+num2;
                break;
        case '-': res=num1-num2;
                break;
        case '*': res=num1*num2;
                break;
        case '/': res=num1/num2;
                break;
        case '%': res=num1%num2;
                break;
        case '^': res=pow(num1,num2);
                break;
    }
    push(st,res);
}

int gettype(char c)
{
    switch(c)
    {
        case '+':
        case '-':
        case '*':
        case '/':
        case '^':
        case '%': return OPERATOR;
        default : return OPERAND;
    }
}

```

Prelab Questions:-

- 1) How to evaluate post fix and prefix expressions.

Lab assignment:-

- 1) Evaluate $9 - ((3 * 4) + 8) / 4$
- 2) Evaluate $9 \ 3 \ 4 \ * \ 8 \ + \ 4 \ / \ -$
- 3) Evaluate $+ \ - \ 9 \ 2 \ 7 \ * \ 8 \ / \ 4 \ 12$

Post lab Questions:-

- 1) Which data structure is used to evaluate post fix and prefix expressions.
- 2) Before going to evaluate , is it necessary to check whether given expression is valid expression or not.

EXPERIMENT-4 (Queues)

Aim: Program to implement queue using arrays

Source code:

```
#include<stdio.h>
#define SIZE 10

void enQueue(int);
void deQueue();
void display();

int queue[SIZE], front = -1, rear = -1;

void main()
{
    int value, choice;
    while(1){
        printf("\n\n***** MENU *****\n");
        printf("1. Insertion\n2. Deletion\n3. Display\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&choice);
        switch(choice){
            case 1: printf("Enter the value to be insert: ");
                    scanf("%d",&value);
                    enQueue(value);
                    break;
            case 2: deQueue();
                    break;
            case 3: display();
                    break;
            case 4: display();break;
            default: printf("\nWrong selection!!! Try again!!!");
        }
    }
}
```

```

}
void enqueue(int value){
    if(rear == SIZE-1)
        printf("\nQueue is Full!!! Insertion is not possible!!!");
    else{
        if(front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        printf("\nInsertion success!!!");
    }
}
void dequeue(){
    if(front == rear)
        printf("\nQueue is Empty!!! Deletion is not possible!!!");
    else{
        printf("\nDeleted : %d", queue[front]);
        front++;
        if(front == rear)
            front = rear = -1;
    }
}
void display(){
    if(rear == -1)
        printf("\nQueue is Empty!!!");
    else{
        int i;
        printf("\nQueue elements are:\n");
        for(i=front; i<=rear; i++)
            printf("%d\t", queue[i]);
    }
}
}

```

Prelab Questions:-

- 1) What is a Queue
- 2) What are the operations of queue
- 3) What are the applications of queue

Lab assignment:-

- 1) Write a program to implement queue using two stacks

Post lab Questions:-

- 1) What are the differences between stack and queue
- 2) What are conditions for queue is full
- 3) What are the conditions for queue is empty.

Aim: Program to implement queue using Linked List**Source code:**

```

#include<stdio.h>
#include<stdlib.h>

struct Node{
    int data;
    struct Node *next;
}*front = NULL,*rear = NULL,*newNode;

void ins(int);
void del();
void display();

void main()
{
    int ch,value,c=0;
    while(!c){
        printf("1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&ch);
        switch(ch){
            case 1: printf("Enter the value to be insert: ");
                    scanf("%d", &value);
                    ins(value);
                    break;
            case 2: del(); break;
            case 3: display(); break;
            case 4: c++;display();break;
            default: printf("\nWrong selection!!! Please try again!!!\n");
        }
    }
}

void ins(int value)
{
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if(front == NULL)
        front = rear = newNode;
    else{
        rear->next = newNode;
        rear = newNode;
    }
}

void del()

```

```

{
    if(front == NULL)
        printf("\nQueue is Empty!!!\n");
    else{
        struct Node *temp = front;
        front = front -> next;
        printf("\nDeleted element: %d\n", temp->data);
        free(temp);
    }
}
}
void display()
{
    if(front == NULL)
        printf("\nQueue is Empty!!!\n");
    else{
        struct Node *temp = front;
        while(temp->next != NULL){
            printf("%d--->", temp->data);
            temp = temp -> next;
        }
        printf("%d \n", temp->data);
    }
}
}

```

Prelab Questions:-

- 1) What are the drawbacks of queue using array
- 2) How to connect two nodes in linked list.
- 3) What is a linked queue

Lab assignment:-

- 1) Write a program to implement linked queue using two stacks

Post lab Questions:-

- 1) What are the differences between queue using arrays and queue using linked list.
- 2) Is linked queue is full.

Aim: Program to implement circular queue.**Source code:**

```

#include<stdio.h>
#define SIZE 1000
void enQueue(int);
void deQueue();
void display();
int cQueue[SIZE], front = -1, rear = -1;
void main()
{
    int ch,c=0,dta;
    while(1){
        printf("1. Insert\n2. Delete\n3. Display\n4. Exit\n");
    }
}

```

```

printf("Enter your choice : ");
scanf("%d",&ch);
switch(ch){
    case 1:
        printf("Enter the data to be insert : ");
        scanf("%d",&dta);
        enqueue(dta);
        break;
    case 2: dequeue();
        break;
    case 3: display();
        break;
    case 4: display();c++;break;
    default: printf("Select correct choice.\n");
}
}
}
void enqueue(int value)
{
    if((front == 0 && rear == SIZE - 1) || (front == rear+1))
        printf("Circular Queue is Full.\n");
    else{
        if(rear == SIZE-1 && front != 0)
            rear = -1;
        cQueue[++rear] = value;
        if(front == -1)
            front = 0;
    }
}
void dequeue()
{
    if(front == -1 && rear == -1)
        printf("Circular Queue is Empty.\n");
    else{
        printf("Deleted element : %d\n",cQueue[front++]);
        if(front == SIZE)
            front = 0;
        if(front-1 == rear)
            front = rear = -1;
    }
}
void display()
{
    if(front == -1)
        printf("Circular Queue is Empty.\n");
    else{
        int i = front;

```



```

printf("Circular Queue Elements are : ");
if(front <= rear){
    while(i <= rear)
        printf("%d ",cQueue[i++]);
}
else{
    while(i <= SIZE - 1)
        printf("%d ", cQueue[i++]);
    i = 0;
    while(i <= rear)
        printf("%d ",cQueue[i++]);
}
printf("\n");
}
}

```

Prelab Questions:-

- 1) What are the drawbacks of queue using array
- 2) What is circular queue

Lab assignment:-

- 1) Write a program to implement DFS

Post lab Questions:-

- 1) What are the conditions to check circular queue is full
- 2) What are the conditions to display in circular queue.

Aim: Program to implement queue which permits insertion and deletion at both the ends using arrays

Source code:

```

#include<stdio.h>
#define SIZE 1000
void e_at_r(int);
void e_at_f(int);
int d_at_f();
int d_at_r();
void display();

int queue[SIZE];
int rear=0,front=0,item;

int main()
{
    int ch,dta,c=0;
    while (!c){
        printf("1. Enqueue at front\n2. Enqueue at rear\n");
        printf("3. Dequeue at front\n4. Dequeue at rear\n");
        printf("5. Display\n6. Exit\n");
    }
}

```

```

printf("Enter your choice:");
scanf("%d", &ch);
if(ch==1 || ch==2){
    printf("Enter the data to insert:");
    scanf("%d", &dta);
}
switch (ch){
    case 1:
        e_at_f(dta);break;
    case 2:
        e_at_r(dta);break;
    case 3:
        d_at_f();break;
    case 4:
        d_at_r();break;
    case 5:
        display();break;
    case 6:
        display();c++;break;
    default:
        printf("Select correct Choice.\n");
        break;
}
}
}

void e_at_r(int value)
{
    if(front == SIZE/2)
        printf("Queue is full.\n");
    else{
        queue[front] = value;
        front++;
    }
}

void e_at_f(int value)
{
    if(front==SIZE/2)
        printf("Queue is full.\n");
    else{
        rear--;
        queue[rear] = value;
    }
}
}
int d_at_r()
{

```

```

    if(front == rear){
        printf("Queue is Empty.\n");
        return 0;
    }
    front--;
    item=queue[front+1];
    return item;
}
int d_at_f()
{
    if(front == rear){
        printf("Queue is Empty.\n");
        return 0;
    }
    rear++;
    item=queue[rear-1];
    return item;
}

void display()
{
    int i;
    if(front == rear)
        printf("Queue is Empty.\n");
    else{
        printf("The Queue elements are:");
        for(i=rear; i < front; i++)
        {
            printf("%d ",queue[i]);
        }
    }
    printf("\n");
}

```

Prelab Questions:-

- 1) What is priority Deque
- 2) What are the differences between queue and deque

Lab assignment:-

- 1) Write a program to implement MAX Heap
- 2) Write a program to check whether the given or entered string is palindrome or not, Implement using Queue.

Post lab Questions:-

- 1) Write the advantages of double ended queue over queue

Aim: Program to implement queue which permits insertion and deletion at both the ends using Linked List

Source code:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int dta;
    struct node *prv, *nxt;
} *st=NULL, *en=NULL, *nw, *tmp;
struct node *create(int dta){
    nw = (struct node*)malloc(sizeof (struct node));
    nw->dta = dta;
    nw->nxt = nw->prv = NULL;
    return (nw);
}
void create_in() {
    st = create(0);
    en = create(0);
    st->nxt = en;
    en->prv = st;
}
void e_at_f(int dta) {
    nw = create(dta);
    tmp = st->nxt;
    st->nxt = nw;
    nw->prv = st;
    nw->nxt = tmp;
    tmp->prv = nw;
}
void e_at_r(int dta) {
    nw=create(dta);
    tmp = en->prv;
    en->prv = nw;
    nw->nxt = en;
    nw->prv = tmp;
    tmp->nxt = nw;
}
void d_at_f() {
    if (st->nxt==en)
        printf("Queue is empty\n");
    else {
        tmp=st->nxt;
        st->nxt=tmp->nxt;
        tmp->nxt->prv=st;
        free(tmp);
    }
    return;
}
void d_at_r(){
```

```

    if (en->prv == st)
        printf("Queue is empty\n");
    else {
        tmp = en->prv;
        en->prv = tmp->prv;
        tmp->prv->nxt = en;
        free(tmp);
    }
    return;
}

void print() {
    if (st->nxt==en){
        printf("Queue is empty\n");
        return;
    }
    tmp=st->nxt;
    while (tmp!=en){
        printf("%d ",tmp->dta);
        tmp=tmp->nxt;
    }
    printf("\n");
}

void main(){
    int dta, ch,c=0;
    create_in();
    while (!c){
        printf("1. Enqueue at front\n2. Enqueue at rear\n");
        printf("3. Dequeue at front\n4. Dequeue at rear\n");
        printf("5. Display\n6. Exit\n");
        printf("Enter your choice:");
        scanf("%d", &ch);
        if(ch==1 || ch==2){
            printf("Enter the dta to insert:");
            scanf("%d", &dta);
        }
        switch (ch){
            case 1:
                e_at_f(dta);break;
            case 2:
                e_at_r(dta);break;
            case 3:
                d_at_f();break;
            case 4:
                d_at_r();break;
            case 5:
                print();break;
            case 6:
                print();c++;break;
            default:
                printf("Select correct Choice.\n");
        }
    }
}

```

```

        break;
    }
}
}

```

Prelab Questions:-

- 1) What is priority Deque
- 2) What are the differences between queue and deque

Lab assignment:-

- 1) Write a program to implement MIN Heap
- 2) Write a program to check whether the given or entered string is palindrome or not, Implement using Queue.

Post lab Questions:-

- 1) Write the advantages of double ended queue over queue
- 2) What are the conditions for dequeu is empty.

EXPERIMENT-5 (Binary Search Tree)

Aim: Program to create Binary Search Tree and perform insertion, deletion and 3 traversal operations

Source code:

```

#include<stdio.h>
#include<stdlib.h>
struct node *insert(struct node *,int );
struct node *delet(struct node *,int );
void traversal();
void inorder();
void preorder();
void postorder();
struct node
{
    int data;
    struct node *right;
    struct node *left;
};
struct node *root;
int c;
void main()
{
    int ch,ele,val;
    struct node *nn;
    printf("\n1.Insert\n2.Delete\n3.Traversal\n4.Exit\n");
    while(ch!=4)
    {
        printf("Enter choice(1 to 4):");
        scanf("%d",&ch);
        switch(ch)

```

```

        {
            case 1:
                printf("Enter data:");
                scanf("%d",&ele);
                root=insert(root,ele);
                break;
            case 2:
                printf("\n Enter the element to be deleted : ");
                scanf("%d",&val);
                root=delet(root,val);
                break;
            case 3:
                traversal();
                break;
            case 4:
                printf("\n\tEXIT\t\n");
                break;
            default:
                printf("Enter correct choice");
                break;
        }
    }
}

struct node *insert(struct node *root, int ele)
{
    struct node *temp,*nn,*p;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=ele;
    nn->left=NULL;
    nn->right=NULL;
    if(root==NULL)
    {
        root=nn;
        root->left=NULL;
        root->right=NULL;
    }
    else
    {
        temp=NULL;
        p=root;
        while(p!=NULL)
        {
            temp=p;
            if(ele<p->data)
            {
                p=p->left;
            }
        }
    }
}

```

```

        else
        {
            p=p->right;
        }
    }
    if(ele<temp->data)
    {
        temp->left=nn;
    }
    else
    {
        temp->right=nn;
    }
}
return root;
}
void traversal()
{
    int a;
    printf("\n1.Preorder\n2.Inorder\n3.Postorder\n4.Exit\n");
    while(a!=4)
    {
        printf("Enter choice(1 to 4):");
        scanf("%d",&a);
        switch(a)
        {
            case 1:
                preorder(root);
                break;
            case 2:
                inorder(root);
                break;
            case 3:
                postorder(root);
                break;
            case 4:
                printf("\n\tEXIT\n");
                break;
            default:
                printf("Enter correct option");
                break;
        }
    }
}
void preorder(struct node *root)
{
    if(root!=NULL)

```



```

        {
            printf("%d\t",root->data);
            preorder(root->left);
            preorder(root->right);
        }
    }
void inorder(struct node *root)
{
    if(root!=NULL)
    {
        inorder(root->left);
        printf("%d\t",root->data);
        inorder(root->right);
    }
}
void postorder(struct node *root)
{
    if(root!=NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d\t ",root->data);
    }
}
struct node *delet(struct node *root,int val)
{
    struct node *cur,*parent,*suc,*psuc,*ptr;
    if(root->left==NULL)
    {
        printf("\n The tree is empty ");
        return(root);
    }
    parent=root;
    cur=root->left;
    while(cur!=NULL&&val!=cur->data)
    {
        parent=cur;
        cur=(val<cur->data)?cur->left:cur->right;
    }
    if(cur==NULL)
    {
        printf("\n The value to be deleted is not present in the tree\n");
        return(root);
    }
    if(cur->left==NULL)
        ptr= cur->right;
    else if(cur->right==NULL)

```

```

        ptr=cur->left;
    else
    {
        psuc=cur;
        cur=cur->left;
        while(suc->left!=NULL)
        {
            psuc=suc;
            suc=suc->left;
        }
        if(cur==psuc)
        {
            suc->left=cur->right;
        }
        else
        {
            suc->left=cur->left;
            psuc->left=suc->right;
            suc->right=cur->right;
        }
        ptr=suc;
    }
    if(parent->left==cur)
        parent->left=ptr;
    else
        parent->right=ptr;
    free(cur);
    return root;
}

```

Prelab Questions:-

- 1) What is a Binary Tree
- 2) What is a Binary Search Tree
- 3) What are the different types of trees
- 4) What are the properties of BST

Lab assignment:-

- 1) Construct a BST for 45,39,56,12,34,78,32,10,89,54,67,81

Post lab Questions:-

- 1) What are different types of tree traversal techniques are there
- 2) What are the applications of BST

Aim: Program to count number of nodes of a Binary Search Tree**Source code:**

```

#include<stdio.h>
#include<stdlib.h>
struct node *insert(struct node *,int );

```

```

void count();
struct node
{
    int data;
    struct node *right;
    struct node *left;
};
struct node *root;
int c=0,e;
main()
{
    int ch,ele;
    struct node *nn;
    printf("\n1.Insert\n2.Traversal\n3.Exit\n");
    while(ch!=3)
    {
        printf("Enter choice(1 to 3):");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter data:");
                scanf("%d",&ele);
                root=insert(root,ele);
                break;
            case 2:
                count(root);
                printf("The count is %d\n",c);
                break;
            case 3:
                printf("\n\tEXIT\t\n");
                break;
            default:
                printf("Enter correct choice");
                break;
        }
    }
}

struct node *insert(struct node *root, int ele)
{
    struct node *temp,*nn,*p;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=ele;
    nn->left=NULL;
    nn->right=NULL;
    if(root==NULL)

```

```

    {
        root=nn;
        root->left=NULL;
        root->right=NULL;
    }
else
{
    temp=NULL;
    p=root;
    while(p!=NULL)
    {
        temp=p;
        if(ele<p->data)
        {
            p=p->left;
        }
        else
        {
            p=p->right;
        }
    }
    if(ele<temp->data)
    {
        temp->left=nn;
    }
    else
    {
        temp->right=nn;
    }
}
return root;
}
void count(struct node *root)
{
    if(root!=NULL)
    {
        count(root->left);
        count(root->right);
        c++;
    }
}

```

Prelab Questions:-

- 1) How many nodes are present in a tree of height h

Lab assignment:-

- 1) Write a program to check whether a node is having zero or one or two children

Post lab Questions:-

- 1) What is the recurrence relation to check total number of nodes in a BST

Aim: Program to find nth node in the InOrder traversal of a BST**Source code:**

```
#include <stdio.h>
#include <stdlib.h>

/* A binary tree node has data, pointer to left child
and a pointer to right child */
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

/* Helper function that allocates a new node with the
given data and NULL left and right pointers. */
struct Node* newNode(int data)
{
    struct Node* node =
        (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;

    return (node);
}

/* Given a binary tree, print its nth nodes of inorder*/
void NthInorder(struct Node* node, int n)
{
    static int count = 0;
    if (node == NULL)
        return;

    if (count <= n) {

        /* first recur on left child */
        NthInorder(node->left, n);
        count++;

        // when count = n then print element
        if (count == n)

```

```

        printf("%d ", node->data);

        /* now recur on right child */
        NthInorder(node->right, n);
    }
}

/* Driver program to test above functions*/
void main()
{
    struct Node* root = newNode(10);
    root->left = newNode(20);
    root->right = newNode(30);
    root->left->left = newNode(40);
    root->left->right = newNode(50);

    int n;
    printf("Enter element position : ");
    scanf("%d",&n);
    NthInorder(root, n);
}

```

Prelab Questions:-

- 1) What is InOrder Traversal technique

Lab assignment:-

- 1) Find the postorder tree traversal for the given InOrder D B E A F C G and Preorder A B D E C F G

Post lab Questions:-

- 1) What is a complete binary tree

Aim: Program to find largest and smallest elements of a BST**Source code:**

```

#include<stdio.h>
#include<stdlib.h>
struct node *insert(struct node *,int );
struct node *small(struct node *root);
struct node *large(struct node *root);
struct node
{
    int data;
    struct node *right;
    struct node *left;
};
struct node *root;

```

```

main()
{
    int ch,ele;
    struct node *nn;
    printf("\n1.Insert\n2.Small\n3.Large\n4.Exit\n");
    while(ch!=4)
    {
        printf("Enter choice(1 to 4):");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter data:");
                scanf("%d",&ele);
                root=insert(root,ele);
                break;
            case 2:
                nn=small(root);
                printf("\nThe Small element is %d\n",nn->data);
                break;
            case 3:
                nn=large(root);
                printf("\nThe large element is %d\n",nn->data);
                break;
            case 4:
                printf("\n\tEXIT\t\n");
                break;
            default:
                printf("\nEnter correct choice");
                break;
        }
    }
}

struct node *insert(struct node *root, int ele)
{
    struct node *temp,*nn,*p;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=ele;
    nn->left=NULL;
    nn->right=NULL;
    if(root==NULL)
    {
        root=nn;
        root->left=NULL;
        root->right=NULL;
    }
    else

```

```

{
    temp=NULL;
    p=root;
    while(p!=NULL)
    {
        temp=p;
        if(ele<p->data)
        {
            p=p->left;
        }
        else
        {
            p=p->right;
        }
    }
    if(ele<temp->data)
    {
        temp->left=nn;
    }
    else
    {
        temp->right=nn;
    }
}
return root;
}
struct node *small(struct node *root)
{
    if(root==NULL || root->left==NULL)
    {
        return root;
    }
    else
    {
        return small(root->left);
    }
}
struct node *large(struct node *root)
{
    if(root==NULL || root->right==NULL)
    {
        return root;
    }
    else
    {
        return large(root->right);
    }
}

```


}

Prelab Questions:-

- 1) Where will be the smallest and largest element will present in BST

Lab assignment:-

- 1) Construct BST for the keys 98,2, 48, 12, 56, 32, 4, 67, 23, 87, 23, 55, 46 and find the smallest and largest keys.

Post lab Questions:-

- 1) What is condition to check searching key is present on the left side or right side.

Aim: Program to find all the elements of Nth level**Source code:**

```
#include<stdio.h>
#include<stdlib.h>
struct node *insert(struct node *,int );
void level(struct node *root,int l,int n);
struct node
{
    int data;
    struct node *right;
    struct node *left;
};
struct node *root;
int n=0,l;
main()
{
    int ch,ele;
    struct node *nn;
    printf("\n1.Insert\n2.Level\n3.Exit\n");
    while(ch!=3)
    {
        printf("Enter choice(1 to 3):");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter data:");
                scanf("%d",&ele);
                root=insert(root,ele);
                break;
            case 2:
                printf("Enter level:");
                scanf("%d",&l);
                level(root,l,n);
```

```

        printf("\n");
        break;
    case 3:
        printf("\n\tEXIT\t\n");
        break;
    default:
        printf("Enter correct choice");
        break;
    }
}
}
struct node *insert(struct node *root, int ele)
{
    struct node *temp,*nn,*p;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=ele;
    nn->left=NULL;
    nn->right=NULL;
    if(root==NULL)
    {
        root=nn;
        root->left=NULL;
        root->right=NULL;
    }
    else
    {
        temp=NULL;
        p=root;
        while(p!=NULL)
        {
            temp=p;
            if(ele<p->data)
            {
                p=p->left;
            }
            else
            {
                p=p->right;
            }
        }
        if(ele<temp->data)
        {
            temp->left=nn;
        }
        else
        {

```

```

        temp->right=nn;
    }
}
return root;
}
void level(struct node *root,int l,int n)
{
    if(root==NULL)
    {
        return;
    }
    else if(l==n)
    {
        printf("%d\t",root->data);
    }
    else
    {
        level(root->left,l,n+1);
        level(root->right,l,n+1);
    }
}
}

```

Prelab Questions:-

- 1) What is level order traversal
- 2) How many nodes are present in the nth level

Lab assignment:-

- 1) Construct BST for the keys 98,2, 48, 12, 56, 32, 4, 67, 23, 87, 23, 55, 46 and find the 3rd level elements

Post lab Questions:-

- 1) Can we implement graphs using level order traversal

Aim: Program to find nodes which are at max distance from the root in a Binary Search Tree

Source code:

```

#include<stdio.h>
#include<stdlib.h>
struct node *insert(struct node *,int );
void level(struct node *root,int l,int n);
int height(struct node *root);
struct node
{
    int data;
    struct node *right;
    struct node *left;
};

```

```

struct node *root;
int n=0,l;
main()
{
    int ch,ele;
    struct node *nn;
    printf("\n1.Insert\n2.Height\n3.Exit\n");
    while(ch!=3)
    {
        printf("Enter choice(1 to 3):");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("Enter data:");
                scanf("%d",&ele);
                root=insert(root,ele);
                break;
            case 2:
                printf("Height is %d\n",height(root));
                printf("Distant node from the root:");
                level(root,height(root)-1,n);
                printf("\n");
                break;
            case 3:
                printf("\n\tEXIT\t\n");
                break;
            default:
                printf("Enter correct choice");
                break;
        }
    }
}

struct node *insert(struct node *root, int ele)
{
    struct node *temp,*nn,*p;
    nn=(struct node*)malloc(sizeof(struct node));
    nn->data=ele;
    nn->left=NULL;
    nn->right=NULL;
    if(root==NULL)
    {
        root=nn;
        root->left=NULL;
        root->right=NULL;
    }
}

```

```

else
{
    temp=NULL;
    p=root;
    while(p!=NULL)
    {
        temp=p;
        if(ele<p->data)
        {
            p=p->left;
        }
        else
        {
            p=p->right;
        }
    }
    if(ele<temp->data)
    {
        temp->left=nn;
    }
    else
    {
        temp->right=nn;
    }
}
return root;
}
void level(struct node *root,int l,int n)
{
    if(root==NULL)
    {
        return;
    }
    else if(l==n)
    {
        printf("%d\t",root->data);
    }
    else
    {
        level(root->left,l,n+1);
        level(root->right,l,n+1);
    }
}
int height(struct node *root)
{
    int right_height,left_height;

```

```

if(root==NULL)
{
    return 0;
}
left_height=height(root->left);
right_height=height(root->right);
if(left_height>right_height)
{
    return left_height+1;
}
else
{
    return right_height+1;
}
}

```

Prelab Questions:-

- 1) What is Depth of a tree
- 2) What is height of a tree

Lab assignment:-

- 1) Write a programme to determine total number of internal in a BST

Post lab Questions:-

- 1) What is the recurrence relation for to find height of node

EXPERIMENT-6 (Heaps)

Aim: Program to create MAX Heap**Source code:**

```

#include <stdio.h>
int arr[100],n;
void display()
{
    int i;
    if(n==0)
    {
        printf("Heap is empty\n");
        return;
    }
    for(i=0;i<n;i++)
        printf("%d\t",arr[i]);
    printf("\n");
}
void insert(int num,int loc)
{
    int par;

```

```

while(loc>0)
{
    par=(loc-1)/2;
    if(num<=arr[par])
    {
        arr[loc]=num;
        return;
    }
    arr[loc]=arr[par];
    loc=par;
}
arr[0]=num;
}
void del(int num)
{
    int left,right,i,temp,par;

    for(i=0;i<n;i++)
    {
        if(num==arr[i])
            break;
    }
    if( num!=arr[i] )
    {
        printf("%d not found in heap\n",num);
        return;
    }
    arr[i]=arr[n-1];
    n=n-1;
    par=(i-1)/2;
    if(arr[i] > arr[par])
    {
        insert( arr[i],i);
        return;
    }
    left=2*i+1;
    right=2*i+2;
    while(right < n)
    {
        if( arr[i]>=arr[left] && arr[i]>=arr[right] )
            return;
        if( arr[right]<=arr[left] )
        {
            temp=arr[i];
            arr[i]=arr[left];
            arr[left]=temp;

```

```

        i=left;
    }
    else
    {
        temp=arr[i];
        arr[i]=arr[right];
        arr[right]=temp;
        i=right;
    }
    left=2*i+1;
    right=2*i+2;
}
if( left==n-1 && arr[i]<arr[left] )
{
    temp=arr[i];
    arr[i]=arr[left];
    arr[left]=temp;
}
}
void main()
{
    int choice,num;
    n=0;
    printf("1.Insert\n");
    printf("2.Delete\n");
    printf("3.Display\n");
    printf("4.Quit\n");
    while(choice!=4)
    {
        printf("Enter your choice (1-4): ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                printf("Enter the number to be inserted : ");
                scanf("%d",&num);
                insert(num,n);
                n=n+1;
                break;
            case 2:
                printf("Enter the number to be deleted : ");
                scanf("%d",&num);
                del(num);
                break;
            case 3:
                display();
                break;

```



```

        case 4:
    break;
        default:
            printf("Wrong choice\n");
        }
    }
}

```

Prelab Questions:-

- 1) What is Heap
- 2) What is complete Binary tree
- 3) What is MAX Heap

Lab assignment:-

- 1) Write a programme to implement priority queue using MAX Heap.
- 2) Write a programme to find all leaf nodes in MAX Heap
- 3) Write heap sort on elements 15 6 8 12 14 16

Post lab Questions:-

- 1) What is the formula for to calculate left child of node I if index starts with 1
- 2) What is the formula for to calculate right child of node I if index starts with 1
- 3) What is the parent of I if index starts with 1.

Aim: Program to create MIN Heap**Source code:**

```

#include <stdio.h>
int arr[100],n;
void display(){
    int i;
    if(n==0){
        printf("Heap is empty\n");
        return;
    }
    for(i=0;i<n;i++){
        printf("%d ",arr[i]);
        printf("\n");
    }
}
void insert(int num,int loc)
{
    int par;
    while(loc>0)
    {
        par=(loc-1)/2;
        if(num>=arr[par])
        {
            arr[loc]=num;
            return;
        }
    }
}

```

```

    }
    arr[loc]=arr[par];
    loc=par;
}
arr[0]=num;
}
void del(int num)
{
    int left,right,i,temp,par;

    for(i=0;i<n;i++)
    {
        if(num==arr[i])
            break;
    }
    if( num!=arr[i] )
    {
        printf("%d not found in heap\n",num);
        return;
    }
    arr[i]=arr[n-1];
    n=n-1;
    par=(i-1)/2;
    if(arr[i] > arr[par])
    {
        insert( arr[i],i);
        return;
    }
    left=2*i+1;
    right=2*i+2;
    while(right < n)
    {
        if( arr[i]>=arr[left] && arr[i]>=arr[right] )
            return;
        if( arr[right]<=arr[left] )
        {
            temp=arr[i];
            arr[i]=arr[left];
            arr[left]=temp;
            i=left;
        }
        else
        {
            temp=arr[i];
            arr[i]=arr[right];
            arr[right]=temp;

```

```

        i=right;
    }
    left=2*i+1;
    right=2*i+2;
}
if( left==n-1 && arr[i]<arr[left] )
{
    temp=arr[i];
    arr[i]=arr[left];
    arr[left]=temp;
}
}
void main()
{
    int choice,num;
    n=0;
    printf("1.Insert\n");
    printf("2.Delete\n");
    printf("3.Display\n");
    printf("4.Quit\n");
    while(choice!=4)
    {
        printf("Enter your choice (1-4); ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                printf("Enter the number to be inserted : ");
                scanf("%d",&num);
                insert(num,n);
                n=n+1;
                break;
            case 2:
                printf("Enter the number to be deleted : ");
                scanf("%d",&num);
                del(num);
                break;
            case 3:
                display();
                break;
            case 4:
                break;
            default:
                printf("Wrong choice\n");
        }
    }
}

```

Prelab Questions:-

- 1) What are the properties of Heap Tree
- 2) What is MIN Heap

Lab assignment:-

- 1) Write a programme to implement priority queue using MIN Heap
- 2) Write a programme to implement HEAP Sort algorithm

Post lab Questions:-

- 1) Is Min Heap is a Complete Binary Tree
- 2) What is the formula to calculate all leaf nodes in Min Heap
- 3) What is the time complexity of HEAP Sort

EXPERIMENT-7 (Hashing)

Aim: Program to search for an element in a hash table

Source code:

```
#include<stdio.h>
#define size 5
int arr[size];
void init()
{
    int i;
    for(i = 0; i < size; i++)
        arr[i] = -1;
}
void insert(int value){
    int key = value % size;
    if(arr[key] == -1){
        arr[key] = value;
        printf("%d inserted at arr[%d]\n", value, key);
    }
    else{
        printf("Collision : arr[%d] has element %d already!\n", key, arr[key]);
        printf("Unable to insert %d\n", value);
    }
}
void del(int value){
    int key = value % size;
```

```

    if(arr[key] == value)
        arr[key] = -1;
    else
        printf("%d not present in the hash table\n",value);
}
void search(int value)
{
    int key = value % size;
    if(arr[key] == value)
        printf("Search Found\n");
    else
        printf("Search Not Found\n");
}
void print(){
    int i;
    printf("-----\n");
    printf("\tLoc - Dta\t\n");
    printf("-----\n");
    for(i = 0; i < size; i++)
        printf("\t%d \t%d\t\n",i,arr[i]);
    printf("-----\n");
}
void main(){
    init();
    int ch,x,c=0;
    printf("\n1.Insert\n2.Search\n3.Delete\n4.Display\n");
    while(!c){
        printf("Enter choice : ");
        scanf("%d",&ch);
        switch(ch){
            case 1:
                printf("Enter element to insert : ");
                scanf("%d",&x);insert(x);break;
            case 2:
                printf("Enter element to search : ");
                scanf("%d",&x);search(x);break;
            case 3:
                printf("Enter element to delete : ");
                scanf("%d",&x);del(x);break;
            case 4:
                print();c++;break;
        }
    }
}

```

Prelab Questions:

- 1) What is hashing
- 2) What are different types of hash functions
- 3) What is linear probing, quadratic probing, double hashing and chaining

Lab assignment:-

- 1) Consider a hash table of size 10. Using linear probing, quadratic probing, double hashing and chaining insert the keys 72, 27, 36, 24, 63, 81, 92 and 101

Post lab Questions:-

- 1) Which open addressing technique is free from clustering problems
- 2) What are the applications of Hashing

Content Beyond Syllabi

Experiment-8

Aim: Program to insert a node after the nth node from the right end

Source code:

```
#include<stdio.h>
#include<stdlib.h>
static int c=0;
struct node
{
    int data;
    struct node *next;
}*start=NULL;
int create()
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    printf("Enter Data , -1 to stop:");
    scanf("%d",&temp->data);
    if(temp->data!=-1)
    {
        temp->next=NULL;
        if(start==NULL)
            start=temp;
        else
        {
            struct node *p=start;
            while(p->next!=NULL)
                p=p->next;
            p->next=temp;
        }
        c++;create();
    }
}
int display() //Displaying Content
{
    if(start==NULL)
        printf("List Is Empty\n");
    else
    {
        struct node *p=start;
        while(p->next!=NULL)
        {
            printf("%d\t",p->data);
            p=p->next;
        }
    }
}
```



```

    }
    printf("%d\t",p->data);
}
}
int IB()//At Beginning
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    printf("Enter Data:");
    scanf("%d",&temp->data);
    temp->next=start;
    start=temp;c++;
}
int IN(int n)
{
    int i;
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    printf("\nEnter Data:");
    scanf("%d",&temp->data);
    temp->next=NULL;
    int L=c;
    if(n>L)
        printf("Not Possible");
    else
    {
        int v;
        struct node *p;
        p=start;
        for(i=0;i<n-2;i++)//Before Nth Node
            p=p->next;
        temp->next=p->next;
        p->next=temp;
    }
    c++;
}
int Rev()
{
    int i=0,j=c-1,k;
    struct node *p,*q,*temp;
    temp=(struct node *)malloc(sizeof(struct node));
    p=q=start;
    if(start==NULL)
        printf("List Is Empty\n");
    else
    {

```

```

while(i<j)
{
    k=0;
    while(k<j)
    {
        q=q->next;
        k++;
    }
    temp->next=NULL;
    temp->data=p->data;
    p->data=q->data;
    q->data=temp->data;
    i++;j--;
    q=start;p=p->next;
}
}
}
int main()
{
    label:
    printf("\n----select the option \n1.Create\n2.Display\n3.Insert\n4.Exit");int m;
    scanf("%d",&m);
    switch(m)
    {
        case 1:create();goto label;
        case 2:display();goto label;
        case 3:{
            printf("\nEnter Nth Node Value:");int n;
            scanf("%d",&n);
            if(n<=c)
            {
                if(n>1)
                {
                    Rev();
                    IN(n);
                    Rev();
                }
                else if(n==1)
                {
                    Rev();
                    IB();
                    Rev();
                }
            }
            else
            {
                printf("Not Possible\n");
            }
        }
    }
}

```

```

        goto label;
    }
}
else
{
    printf("Not Possible");
    goto label;
}
goto label;
}
    case 4:exit(0);
    default:printf("Enter Valid Option");
        goto label;
    }
}
}

```

Experiment-9

Aim: Program to delete all duplicate elements from single linked list

Source code:

```

Delete duplicate:
#include<stdio.h>
#include<stdlib.h>
static int c=0;
struct node
{
    int data;
    struct node *next;
}*start=NULL;
int create()
{
    struct node *temp;
    temp=(struct node *)malloc(sizeof(struct node));
    printf("Enter Data(-1 To Stop):");
    scanf("%d",&temp->data);
    if(temp->data!=-1)
    {
        temp->next=NULL;
        if(start==NULL)
            start=temp;
        else
        {
            struct node *p=start;

```

```

while(p->next!=NULL)
    p=p->next;
p->next=temp;
}
c++;create();
}
}
int display() //Displaying Content
{
    if(start==NULL)
        printf("List Is Empty\n");
    else
    {
        struct node *p=start;
        while(p->next!=NULL)
        {
            printf("%d\t",p->data);
            p=p->next;
        }
        //printf("%d\t",p->data);
    }
}
int dup()
{
    struct node *p=start,*q,*temp;
    while(p->next!=NULL)
    {
        q=p->next;
        while(q->next!=NULL)
        {
            if(p->data==q->data)
            {
                temp=q->next;
                q->next=q->next->next;
                free(temp);c--;
            }
            else
                q=q->next;
        }
        p=p->next;
    }
}
int main()
{
    label:
    printf("\n----Wanna\n1.Create\n2.Display\n3.Delete Duplicate Elements\n4.Exit");int m;

```

```

scanf("%d",&m);
switch(m)
{
case 1:create();goto label;
case 2:display();goto label;
case 3:{
    if(c>0)
    {
        dup();
        goto label;
    }
    else
    {
        printf("\nList is Empty");goto label;
    }
}
case 4:exit(0);goto label;
default:printf("\nEnter Valid Choice");goto label;
}
}

```

Experiment-10

Aim: Program to sort and reverse the values after nth position in single linked list

Source code:

```

#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
typedef struct student
{
    int id;
    struct student *next;
}node;
node *create(int n);
node *reverse(node *start,int n);
void display(node * start);
void afternth(node *start,int);
void sort(node *);
main()
{
    int n;node *start,*m;
    printf("enter the size");
    scanf("%d",&n);

```

```

start=create(n);
afternth(start,n);

/*m=reverse(start,n);
display(m);*/
}
node *create(int n)
{
    int i;
    node *temp,*start=NULL,*ptr;
    for(i=0;i<n;i++)
    {
        temp=(node *)malloc(sizeof(node));
        printf("data==");
        scanf("%d",&temp->id);
        temp->next=NULL;

        if (start==NULL)
        {
            start=temp;
        }
        else
        {
            ptr=start;
            while(ptr->next!=NULL)
            {
                ptr=ptr->next;
            }
            ptr->next=temp;
        }
    }
    return start;
}
node *reverse (node * start,int n)
{
    node *p,*q;int t,i=0,j=n-1;
    int k;q=start,p=start;
    while(i<j)
    {k=0;
    while(k<j)
    {
        q=q->next;
        k++;
    }
    t=p->id;
    p->id=q->id;
    q->id=t;

```

```

i++;j--;
p=p->next;
q=start;
}
return start;

}
void display(node * start)
{
printf("elements in the linke list are::\n");
node *ptr; ptr=start;
if(start==NULL)
{
printf("\n There are no elements in the list\n");
}

else
{
while(ptr!=NULL)
{
printf("%d->",ptr->id);
ptr=ptr->next;
}printf("NULL");
}
}
void afrenrth(node * start,int k)
{
int n;
printf("enter n value:\n");
scanf("%d",&n);
node *ptr,*m; ptr=start;
int i=0;
while(ptr!=NULL)
{
if(i==n)
{
sort(ptr);
printf("\nafter reversing\n");
m=reverse(ptr,k-n+1);
display(m);
printf("\nafter sorting\n");
sort(ptr);
display(m);
}
}
}

```

```

ptr=ptr->next;
i++;
}

}

void sort(node *start)
{
node *ptr,*j;int l;
ptr=start;
for(ptr=start;ptr->next != NULL ;ptr=ptr->next)
{
for(j=ptr->next ;j!=NULL ;j=j->next)
{
if(ptr->id > j->id)
{
l=ptr->id;
ptr->id=j->id;
j->id=l;
}
}
}
}
}

```

Experiment-11

Aim: Program to check whether entered string is a subsequence string of the existing string.

Source code:

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
main()
{
int rear=-1,front=-1,i,data,flag=0,l;char a[15],q[10];
printf("\nEnter the length of the Q");
scanf("%d",&l);
printf("\nEnter the elements into Q");

```



```

for(i=0;i<l;i++)
{
    if(rear==-1)
    {
        front=0;
    }
    rear++;
    scanf(" %c",&data);
    q[rear]=data;
}
printf("\nEnter a string");
scanf(" %s",a);

for(i=0;i<strlen(a);i++)
{
    for(front=0;front<l;front++)
    {
        if(a[i]==q[front])
        {
            flag=flag+1;
            printf("inside flag=%d,,",flag);
            if(flag==strlen(a))
            {
                break;
            }
        } if(flag==strlen(a))
        {
            break;
        }
    }
}

printf("flag=%d",flag);
if(flag == strlen(a))
{
    printf("\nit is a subsequence\n");
}
else
{
    printf("\nits not a subsequence\n");
}
}

```

Experiment-12

Aim: Program to multiply two polynomials

Source code:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int a,b;
    struct node *next;
};
struct node *start=NULL , *f1,*f2 , *start1 = NULL;
int create()
{
    int ct,pow;
    while(1)
    {
        struct node *temp;
        temp=(struct node *)malloc(sizeof(struct node));
        printf("Enter Coefficient , power and Enter -1 to Exit:");
        scanf("%d,%d",&ct,&pow);
        if (ct==-1 || pow ==-1)
            break;
        else
        {
            temp -> a = ct;
            temp -> b = pow;
            temp->next=NULL;
        }
        if(start==NULL)
            start=temp;
        else
        {
            struct node *p;
            p=start;
            while(p->next!=NULL)
                p=p->next;
            p->next=temp;
        }
    }
}
int t(int m,int n)
{
    struct node *t,*s;
    s = (struct node *)malloc(sizeof(struct node));
```

```

t = start1;
s->a = m;
s->b = n;
s->next = NULL;
if (start1 == NULL)
    start1 = s;
else
{
    while (t->next!=NULL)
        t = t->next;
    t->next = s;
}
}
int dis()
{
    struct node *f;
    f = start1;
    while (f->next!=NULL)
    {
        printf("%dx^%d + ",f->a,f->b);
        f= f->next;
    }
    printf("%dx^%d \n",f->a,f->b);
}
int add()
{
    struct node *k1 , *k2 , *k3;
    k1 = start1 ;
    while (k1 != NULL && k1->next != NULL)
    {
        k2 = k1;
        while (k2->next != NULL)
        {
            if (k1->b == k2->next->b)
            {
                k1->a = k1->a + k2->next->a;
                k3 = k2->next;
                k2->next = k2->next->next;
                free(k3);
            }
            else
                k2 = k2->next;
        }
        k1 = k1->next;
    }
}

```

```
}
int mul()
{
    int m,n;
    struct node *p;
    p=f2;
    while (f1!=NULL)
    {
        while (f2!=NULL)
        {
            m = f1->a * f2->a ;
            n = f1->b + f2->b ;
            t(m,n);
            //printf("%dX^%d",m,n);
            f2 = f2->next;
        }
        f2 = p;
        f1 = f1->next;
    }
}
void main()
{
    printf("\n\t Enter First Polynomial \n\n");
    create();
    f1 = start;
    start = NULL;
    printf("\n\t Enter Second Polynomial \n\n");
    create();
    f2 = start;
    start = NULL;
    mul();
    printf("After Multiplication : ");
    add();
    dis();
}
```

Experiment-13

Aim: Program to check whether the given or entered string is palindrome or not, implement using Queue.

Source code:

```
#include<stdio.h>
#include<stdlib.h>
main()
{
    int n,rear=-1,front=-1,q[12],p=0,l=0,i,m;
    printf("\nEnter 10 digits into the Q");
    for(i=0;i<10;i++)
    {
        if(rear== -1)
        {
            front=0;
        }
        rear++;
        scanf("%d",&n);
        q[rear]=n;
        p=p*10+n;
    }
    while(rear < 10 && rear >=0)
    {
        m=q[rear];
        l=l*10+m;
        rear--;
    }
    printf("\nBefore reversing==%d,after reversing==%d\n",p,l);
    if(l==p)
    {
        printf("\nIt is a palindrome\n");
    }
    else
    {
        printf("Not a palindrome");
    }
}
```

Experiment-14

Aim: There are n persons who are arranged in circular passion order (CLL). Find the winner after deleting k -th element respectively.

Source code:

```
#include<stdio.h>
#include<stdlib.h>
typedef struct winner
{

    struct winner *next;
    int per;
    struct winner *prev;
}node;

node * create(node *start);
node *delete(node *,int);
node *deleteatbegin(node *start);
node *deleteatend(node *start);
int count(node *);
main()
{
    int n,i,c,l;node *start=NULL,*m;
    start=create(start);
    printf("\nenter the gap");
    scanf("%d",&n);
    printf("count=%d",count(start));
    for(c=0;count(start)!=1;)
    {
        if(start->prev!=NULL && start->next!=NULL)
        {
            start=delete(start,n);
        }
        else if(start->prev == NULL && start->next != NULL)
        {
            start=deleteatbegin(start);
        }
        else
        {
            start=deleteatend(start);
        }
        printf("\ncount==%d\n",count(start));
    }
    printf("winner =%d",start->per);
```

```

}
```

```

node * create(node *start)
{
    node *temp,*ptr;int n,i;
    printf("\nnno. of persons==");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        temp=(node *)malloc(sizeof(node));
        temp->next=temp->next=NULL;
        printf("\nperson==");
        scanf("%d",&temp->per);
        if(start==NULL)
        {
            start=temp;
        }
        else
        {
            ptr=start;
            while(ptr->next != NULL)
            {
                ptr=ptr->next;
            }
            ptr->next=temp;
            temp->prev=ptr;
        }
    }
    return start;
}
}
```

```

int count(node *start)
{
    node *temp,*ptr;int c=0;
    ptr=start;
    while(ptr->next != NULL)
    {
        c=c+1;
        ptr=ptr->next;
    }
    return c+1;
}
```

```
}

```

```
node *delete(node *start,int pos)
{
    node *ptr,*k;int i;

    ptr=start;
    for(i=0;ptr->next != NULL;i++,ptr=ptr->next)
    {
        if(i==pos)
        {printf("\ni==%d\n",i);

            k=ptr->prev;
            ptr->prev->next=ptr->next;
            ptr->next->prev=ptr->prev;
            free(ptr);
            break;

        }
        ptr=ptr->next;
    }
    return k;
}
```

```
node *deleteatbegin(node *start)
{
    node *ptr;
    ptr=start;
    ptr->next->prev=NULL;
    start=ptr->next;
    free(ptr);
    return start;
}
```

```
node *deleteatend(node *start)
{
    node *ptr,*k;
    ptr=start;
    while(ptr->next != NULL)
    {
        ptr=ptr->next;
    }
}
```



```

}
k=ptr->prev;
ptr->prev->next=NULL;
free(ptr);
return (k);
}

```

Experiment-15

Aim: To Design a calculator application by using either PREFIX or POSTFIX algebraic expression as internal Data structure.

Note: Take the entire input from the user at a time.

Source code:

```

#include<stdio.h>
#include<string.h>
#define MAX 100
int st[MAX],top=-1,flag=0;
void check(char);int push(int);int pop();
int main()
{
    char a[200];
    printf("\n\tThis Is A Calc Application\n");
    printf("Enter Choice\n1.Prefix\t2.Postfix\n");
    switch(getch())
    {
        case '1':{
            printf("Enter Expression(Without Any Extra Characters) : ");gets(a);
            int i=strlen(a)-1;flag=1;
            while(a[i])
            {
                if(isalnum(a[i]))//a[i]>=48 && a[i]<=57
                {
                    push(a[i]-48);
                    i--;
                    continue;
                }
                check(a[i]);
                i--;
            }
            }break;
        case '2':{
            printf("Enter Expression(Without Any Extra Characters) : ");gets(a);
            int i=0;flag=0;

```

```

while(a[i]!='\0')
{
    if(isalnum(a[i])) //a[i]>=48 && a[i]<=57
    {
        push(a[i]-48);
        i++;
        continue;
    }
    check(a[i]);i++;
}
}break;
default:puts("Wrong Choice");exit(0);
}
printf("%d",pop());
}
int push(int v)
{
    if(top==MAX-1)
        printf("Stack Overflow"),exit(0);
    else
        st[++top]=v;
}
int pop()
{
    if(top == -1)
        printf("Invalid Expression"),exit(0);
    else
        return st[top--];
}
void check(char c)
{
    int op1,op2;
    if(flag==1)
        op1=pop(),op2=pop();
    else
        op2=pop(),op1=pop();
    switch(c)
    {
        case '+':push(op1+op2);break;
        case '-':push(op1-op2);break;
        case '*':push(op1*op2);break;
        case '/':push(op1/op2);break;
    }
}

```

Experiment-16

Aim: To Design a Music Player using Circular double linked list

Source code:

```
#include<stdio.h>
#include<conio.h>
struct song {
char song[50];
char movie[50];
struct song* next;
struct song* prev;
}*start=NULL,*control=NULL;
struct song* search(struct song*,char*,char*);
int display_all();
int display(struct song*);
int addSong(char*,char*);
int delSong(char*,char*);
int menustatus=1,playstatus=0;
int main(){
char option,song[50],movie[50];
display_all();
if(menustatus){
printf("Menu:\na)AddSong\nnd)DelSong\nnn)NextSong\nnp)PrevSong\nns)SearchSong\nnr)ReShuffle\nne)Exit\nEnter the option:\n");
menustatus=0;
}
option=getch();
if(option=='a' || option=='d' || option=='s'){
printf("Enter the song and movie:");
scanf("%s%s",song,movie);
}
switch(option){
case 'a':addSong(song,movie);break;
case 'd':delSong(song,movie);break;
case 'n':if(control!=NULL)
control=control->next;
else
printf("Add Songs Mann...");
break;
case 'p':if(control!=NULL)
control=control->prev;
else
printf("Add Songs Mann...");
break;
```

```

case 'r':shuffle();break;
case ' ':if(playstatus)playstatus=0;else {playstatus=1;
    char tmp[100];
    strcpy(tmp,"start ");
    strcat(tmp,control->song);
    strcat(tmp,".mp3");
    system(tmp);
    }break;
case 'e':printf("Wanna exit(y/n):");if(getch()=='y')exit(0);break;
case 's':if(start!=NULL){
    struct song* tmp=search(start,song,movie);
    if(tmp!=NULL){
        int option;
        printf("Menu:\n1)Delete\n2)Play\n");
        scanf("%d",&option);
        switch(option){
            case 1:delSong(song,movie);break;
            case 2:control=tmp;break;
        }
    }
    }break;
}
system("cls");
main();
}
struct song* search(struct song* tmp,char song[50],char movie[50]){
if(start==NULL)
    return NULL;
do
{
    if(strcmp(song,tmp->song)==0 && strcmp(movie,tmp->movie)==0)
        return tmp;
    tmp=tmp->next;
} while(tmp!=start);
return NULL;
}

int addSong(char song[50],char movie[50]){
struct song* new_song=(struct song*)malloc(sizeof(struct song));
strcpy(new_song->song,song);
strcpy(new_song->movie,movie);
if(search(start,song,movie)==NULL){
if(start==NULL){
    start=new_song;
    start->next=start;
    start->prev=start;

```

```

        control=start;
    }
    new_song->prev=start->prev;
    new_song->next=start;
    start->prev=new_song;
    new_song->prev->next=new_song;
    printf("%s is being added\n",start->prev->song);
}
}
int delSong(char song[50],char movie[50]){
    if(start==NULL)
        return 0;
    struct song* del_song=search(start,song,movie);
    printf("%u == %u\n",start,del_song);
    getch();
    if(del_song!=NULL){
        del_song->prev->next=del_song->next;
        del_song->next->prev=del_song->prev;
        if(del_song==start){
            if(start->next==start){ start=NULL;control=NULL;}
            else{
                start=start->next;
                control=start;
            }
        }
        free(del_song);
    }
    else{
        printf("Song Not Found...\n");
    }
}
int display(struct song* tmp){
    if(tmp==NULL){
        printf("No Songs with specified addr\n");
        return 0;
    }
    printf(" /-----\\n");
    if(control==tmp){
        if(playstatus)
            printf(" | playing...t[ %s\t\t\t%s\t\t]t \n",tmp->movie,tmp->song);
        else
            printf(" | paused...t[ %s\t\t\t%s\t\t]t \n",tmp->movie,tmp->song);
    }
    else{
        printf(" | \t\t[ %s\t\t\t%s\t\t]t \n",tmp->movie,tmp->song);
    }
}

```

```

printf(" \-----\n");
}
int display_all(){
struct song* tmp=start;
if(start==NULL)
    return 0;
do
{
    display(tmp);
    tmp=tmp->next;
}while(tmp!=start);
}
int shuffle(){
if(start==NULL || start->next==start)
    return 0;
struct song* p=start;
int count=0;
while(p->next!=start){count++;p=p->next;}
printf("%d\n",count);
for(int j=0;j<count+13;j++){
    struct song* q=p->next;
    int random=rand()%23;
    for(int i=0;i<random;i++)q=q->next;
    /*struct song* tmp1=p->next,*tmp2=p->prev;
    p->next=q->next;
    q->next=tmp1;
    p->prev=q->prev;
    q->prev=tmp2;
    p->next->prev=p;
    p->prev->next=p;
    q->next->prev=q;
    q->prev->next=q;*/
    char tmp[50];
    strcpy(tmp,p->song);
    strcpy(p->song,q->song);
    strcpy(q->song,tmp);
    strcpy(tmp,p->movie);
    strcpy(p->movie,q->movie);
    strcpy(q->movie,tmp);
    p=p->next;
}
}
}

```

Note:- Add the below requirements to the Music player.

Provide movie name for each and every song when user enter letter 'm' you have to display the current song movie name. the user can sort or provide the option to sort the music

player by providing two options 1.movie name 2.song name and then display the all songs accordingly user. When the user enter 'f' you have to move to the first song. When the user enter 'l' you move to the last song

Experiment-17

Aim: To a Program to implement Breadth First Traversal

Source code:

```
#include<stdio.h>
int a[20][20], q[20], visited[20], n, i, j, f = 0, r = -1;

void bfs(int v) {
    for(i = 1; i <= n; i++)
        if(a[v][i] && !visited[i])
            q[++r] = i;
    if(f <= r) {
        visited[q[f]] = 1;
        bfs(q[f++]);
    }
}

void main() {
    int v;
    printf("\n Enter the number of vertices:");
    scanf("%d", &n);

    for(i=1; i <= n; i++) {
        q[i] = 0;
        visited[i] = 0;
    }

    printf("\n Enter graph data in matrix form:\n");
    for(i=1; i<=n; i++) {
        for(j=1;j<=n;j++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("\n Enter the starting vertex:");
    scanf("%d", &v);
    bfs(v);
    printf("\n The node which are reachable are:\n");
```

```

for(i=1; i <= n; i++) {
    if(visited[i])
        printf("%d\t", i);
    else {
        printf("\n Bfs is not possible. Not all nodes are reachable");
        break;
    }
}
}
}

```

Aim: To a Program to implement Depth First Traversal

Source code:

```

#include <stdio.h>
void dfs(int);
int g[10][10],visited[10],n,vertex[10];
void main()
{
    int i,j;
    printf("enter number of vertices:");
    scanf("%d",&n);
    printf("enter the val of vertex:");
    for(i=0;i<n;i++)
        scanf("%d",&vertex[i]);
    printf("\n enter adjacency matrix of the graph:");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            scanf("%d",&g[i][j]);
    for(i=0;i<n;i++)
        visited[i]=0;
    dfs(0);
}
void dfs(int i)
{
    int j;
    printf("%d",vertex[i]);
    visited[i]=1;
    for(j=0;j<n;j++)
        if(!visited[j]&&g[i][j]==1)
            dfs(j);
}

```